Software Requirements Specification

for

Aural Aid: Adaptive AI Hearing with Contextual Understanding

Version v2.0

Prepared by

Name	Roll No.	Email
Ayesha Nazir	**22-SE-079**	**22-SE-079@student.hitecuni.edu.pk**
Hamza Moeen	**22-SE-025**	**22-SE-025@student.hitecuni.edu.pk**

Supervisor Name: Mr. Junaid Ali Khan

Course Name: Software Engineering

Submission Date: 21-01-2026

Table Content

Contents

1. Introduction 1
1.1 Document Purpose 1
1.2 Product Scope 2
1.3 Intended Audience 2
1.4 Definitions, Acronyms, and Abbreviations 3
1.5 Document Conventions 3
1.6 References and Acknowledgments 5
2. Overall Description 5
2.1 Product Overview 6
2.2 Product Functionality 7
2.3 Design and Implementation Constraints 8
2.4 Assumptions and Dependencies 9
3. Specific Requirements 10
3.1 External Interface Requirements 10
3.1.1 User Interfaces 10
3.1.2 Hardware Interfaces 11
3.1.3 Software Interfaces 12
3.2 Functional Requirements 13
3.2.1 User & System Profile Management 13
3.2.2 Real-Time Bionic Transcription Process 13
3.2.3 Biometric Safety Monitoring 14
3.2.4 Safe Navigation & Guardian Mode 15
3.2.5 Centralized Bionic Dashboard 15
3.3 Use Case Model 16
3.3.1 Use Case #1: Power On Device: 16

3.3.2 Use Case #2: Pair Device with Mobile App 17

3.3.3 Use Case #3: User Registration 17

3.3.4. Use Case #4: Sync Device Data 18

3.3.4 Use Case #5: Capture Live Audio 18

3.3.5 Use Case #6: Filter Background Noise 19

3.3.7 Use Case #7: Amplify Speech Signal 20

3.3.8 Use Case #8: Monitor Environmental Noise Level 20

3.3.9 Use Case #9: Trigger Noise Alert 21

3.3.10 Use Case #10: Convert Speech to Text 22

3.3.11 Use Case #11: Display Live Captions 22

3.3.12 Use Case #12: Detect Speaker Emotion 23

3.3.13 Use Case #13: Display Emotion Indicator 24

3.3.14 Use Case #14: Summarize Conversation 24

3.3.15 Use Case #15: Monitor Heart Rate 25

3.3.16 Use Case #16: Monitor SpO_2 Level 26

3.3.17 Use Case #17: Monitor Body Temperature 26

3.3.18 Use Case #18: Detect Abnormal Health Values 27

3.3.19 Use Case #19: Send Health Alert 28

3.3.20 Use Case #20: View Caption History 28

3.3.21 Use Case #21: Update Firmware 29

3.3.22 Use Case #22: Adjust Audio Profile 29

3.3.23 Use Case #23: Emergency SOS Alert 30

3.3.24 Use Case #24: Battery Status Monitoring & Alert 31

4. Other Non-Functional Requirements 32

4.1 Performance Requirements 32

4.1.1 Interview Evaluation Speed 32

4.1.2 Biometric Reporting Speed & Accuracy 32

4.1.3 GUI Responsiveness & System Fluidity 33
4.2 Safety and Security Requirements 33
4.2.1 Edge-Data Privacy and Localized Sovereignty 33
4.2.2 Guardian Mode: Fall Detection and SOS Reliability 34
4.2.3 Hardware Safety and Power Stability 34
4.3 Software Quality Attributes 35
4.3.1 Reliability 35
4.3.2 Adaptability 35
4.3.3 Usability 35
4.3.4 Maintainability 36
4.3.5 Scalability 36
4.3.6 Security 36
5. Design Requirements 37
5.1 High-Level Design 37
5.1.1 Block Diagram 37
5.1.2 Design Architecture Patterns 39
5.2 Structural Design 40
5.2.1 Class Diagram 40
5.2.2 Component Diagram 42
5.2.3 Data Flow Diagrams 44
5.2.4 Activity Diagram: 46
5.2.5 Sequence Diagram: 48
5.2.6 AuralAid Ecosystem: 50
5.2.7 GUI 51
5.1.1 Behavioral Design 58
5.1.2 Use Case Diagram 58
6. Software Planning and Timeline 60

6.1 Work Breakdown Structure 60

6.2 Milestones and Deliverables 68

6.3 Detailed Baseline Plan 69

7. Quality Assurance Plan 71

7.1 Testing Requirements 71

7.1.1 Functional Testing 71

7.1.2 Performance Testing 71

7.1.3 Security Testing 72

7.1.4 Integration Testing 72

7.1.5 Usability Testing 72

7.1.6 Accessibility Testing 72

7.1.7 Compatibility Testing 72

7.1.8 Beta Testing 73

7.2 Acceptance Criteria 73

7.2.1 Functional Acceptance Criteria 73

7.2.2 Performance Acceptance Criteria 73

7.2.3 Security Acceptance Criteria 73

7.2.4 Usability and Accessibility Acceptance Criteria 73

7.2.5 Integration and Compatibility Criteria 73

7.2.6 Reliability and Recovery Criteria 74

7.3 Quality Assurance Plan 74

7.3.1 Planned Test Suites and Test Cases 74

8. Configuration Management Plan 78

1. Introduction

The Auralaid system represents a paradigm shift in assistive technology by integrating a high-performance bionic interface with real-time health surveillance. Built on a dual-computing architecture featuring a Raspberry Pi 4 and an ESP32-S3, the system facilitates advanced features such as OpenAI Whisper transcription for English and Urdu and Adaptive Noise Cancellation to isolate speech in loud environments. This hardware-software synergy ensures that individuals with hearing impairments are not only provided with visual sound representation but are also protected by a Smart Sound Radar and haptic alerts for environmental emergencies like sirens.

Complementing the hardware is a medical-grade Flutter mobile application designed with a high-contrast, OLED-black aesthetic to maximize clarity and battery efficiency. This application functions as a digital companion, rendering live, mathematically smoothed ECG waveforms from the MAX30101 sensor while maintaining a Guardian Mode that uses a 3.5G impact threshold to detect falls and trigger SOS protocols. By merging a precision PCB design with GPS-based safe routing and AI-driven knowledge assistance, Auralaid provides a comprehensive and autonomous ecosystem that actively monitors both the user's biological vitals and their physical surroundings.

1.1 Document Purpose

The purpose of this Software Requirement Specification (SRS) document is to clearly define the requirements, features, and constraints of the Auralaid system. It provides a detailed technical blueprint of how the modular bionic system should behave, specifying the functionalities it must perform—such as real-time transcription and biometric monitoring—and the quality standards it must meet for medical-grade reliability. By documenting hardware pinouts, communication protocols like I2C and I2S, and software architectural patterns, this SRS ensures that all technical components align with the project's assistive goals.

This document acts as a multi-functional reference throughout the project lifecycle:

- It serves as a definitive roadmap for developers to correctly implement complex logic, such as Adaptive Noise Cancellation and Whisper AI inference.
- It provides instructors, reviewers, and stakeholders with a clear standard to understand and verify system expectations.

- It establishes the baseline for system design, PCB routing logic, and rigorous testing and validation protocols.
- It functions as the primary tool for synchronizing technical details and progress between project members and supervisors.

1.2 Product Scope

Auralaid is a modular bionic wearable ecosystem designed to transform the acoustic and biological environment into visual and haptic data for individuals with hearing impairments. Unlike traditional devices that merely amplify sound, this system functions as an intelligent assistant that captures environmental audio, filters interference, and provides comprehensive health and safety monitoring at the edge.

The product aims to:

- Implement Adaptive Noise Cancellation (ANC) using spectral subtraction algorithms to isolate clear signals in noisy environments.
- Utilize OpenAI Whisper (faster-whisper) to generate low-latency speech-to-text for English and Urdu conversations.
- Integrate AI-driven emotional feedback and contextual knowledge assistance via a Wikipedia query engine.
- Provide continuous medical-grade tracking of heart rate, SpO2, and body temperature via a high-sensitivity sensory network.
- Display live captions, ECG waveforms, and safety notifications through a custom Flutter application optimized for OLED-black clarity.
- Visualize safe walking corridors via a "Smart Safety Map" and trigger haptic alerts for emergency sounds or detected falls.
- Utilize low-cost, open-source hardware such as the Raspberry Pi 4 and ESP32-S3 to ensure the technology remains affordable for users in developing regions..

While the system is currently a standalone modular prototype focusing on local edge computing, it is architected with a Modular Micro-Threaded Architecture to allow for future scalability and commercial deployment.

1.3 Intended Audience

This Software Requirement Specification (SRS) is tailored for a diverse group of technical and academic stakeholders involved in the development and validation of the **Auralaid** ecosystem.

The document provides the necessary depth for hardware-software integration while maintaining the high-level clarity required for project oversight.

The intended audience includes:

- Those responsible for implementing the modular micro-threaded architecture, ESP32-S3 firmware, and the Flutter bionic interface.
- Professionals interested in the application of OpenAI Whisper and Adaptive Noise Cancellation (ANC) for individuals with hearing impairments.
- Individuals who will utilize the Test Matrix to verify critical safety features, such as 3.5G fall detection and biometric accuracy.
- Learners studying the integration of real-time sensor networks, PCB routing logic, and Edge-to-Mobile communication protocols.

1.4 Definitions, Acronyms, and Abbreviations

This section defines the key terms, acronyms, and abbreviations used throughout this document. These terms are relevant to the design, implementation, and understanding of the Smart Hearing Aid System project, including technologies like AI, IoT, and DSP, as well as components, software concepts, and interfaces that the system relies on.

Table 1: Definitions, Acronyms, and Abbreviations

Term	Description
AI	Artificial Intelligence
IoT	Internet of Things
DSP	Digital Signal Processing
ESP32-S3	Microcontroller with Wi-Fi and Bluetooth
MEMS	Micro-Electro-Mechanical System microphone
SRS	Software Requirement Specification
API	Application Programming Interface
STT	Speech-to-Text
UI	User Interface
Prototype	Initial working model

1.5 Document Conventions

This document follows the IEEE Software Requirement Specification (SRS) template to ensure clarity, consistency, and professional documentation standards. All requirements are clearly

structured and categorized to avoid ambiguity and improve traceability throughout the development lifecycle.

The requirement keywords used in this document are defined as follows:

Shall:

Indicates mandatory system requirements that must be implemented in the Smart Hearing Aid System.

Should:

Represents recommended features or design practices that enhance system performance but are not strictly mandatory.

Will:

Refers to future enhancements or features that are not part of the current implementation but may be added in later versions.

Formatting Conventions

The following formatting and style guidelines are applied consistently throughout this document to maintain a uniform and professional appearance as show in Table Table 2.

Table 2: Document Convention

Element	Specification
Font	Times New Roman
Heading 1	16pt, Bold
Heading 2	14pt, Bold
Heading 3	12pt, Bold
Body Text	12pt, Justified
Line Spacing	1.5
Paragraph Spacing	12pt after Heading 1
Margins	Standard (Top: 1”, Bottom: 1”, Left: 1”, Right: 1”)
Figure Captions	Positioned below the figures, Default style, Center aligned
Table Captions	Positioned above the tables, Default style, Center aligned
Header	Project Title – Smart Hearing Aid System, right aligned
Footer	Page numbers aligned to the right
Heading Numbering	Limited to three levels (e.g., 1, 1.1, 1.1.1)
Table of Contents	Automatic generation with numbered indexing
List of Figures and Tables	Included only if more than three figures or tables exist
Referencing Style	APA referencing style for all sources

1.6 References and Acknowledgments

[1] World Health Organization, "Deafness and hearing loss," Mar. 2018. [Online]. Available: https://www.who.int/news-room/fact-sheets/detail/deafness-and-hearing-loss

[2] B. Olusanya et al., "Global burden of hearing loss," The Lancet, vol. 393, no. 10175, pp. 912-913, Mar. 2019.

[3] Espressif Systems, "ESP32-S3 Series Datasheet," v1.6, 2024. [Online]. Available: https://www.espressif.com/en/support/documents/technical-documents

[4] TensorFlow, "TensorFlow Lite: ML for mobile and edge devices," 2024. [Online]. Available: https://www.tensorflow.org/lite

[5] Arduino, "Arduino Documentation: Development platform and sensor interfacing," 2024. [Online]. Available: https://docs.arduino.cc/

[6] Google, "Flutter documentation," 2024. [Online]. Available: https://docs.flutter.dev/

[7] G. Klein et al., "faster-whisper: High-performance speech recognition with CTranslate2," 2023. [Online]. Available: https://github.com/SYSTRAN/faster-whisper

2. Overall Description

The Auralaid Intelligent Bionic Hearing and Health Assistant offers a seamless, intelligent, and highly accessible experience for individuals with hearing impairments. By integrating advanced Artificial Intelligence (AI) and Internet of Things (IoT) technologies, the system transcends the capabilities of traditional hearing aids, transforming the acoustic and biological environment into actionable visual and haptic data. Auralaid provides a comprehensive suite of features including real-time speech-to-text transcription for English and Urdu, adaptive noise cancellation, and a medical-grade biometric surveillance network that enhance communication, health monitoring, and environmental safety in daily life.

The system's capabilities are designed to support users at every stage of social and professional interaction, featuring a Smart Sound Radar for environmental warnings and an AI Neural Assistant for contextual knowledge. Engineered with a focus on affordability and edge computing, Auralaid is a standalone, modular solution that operates primarily on local hardware such as the Raspberry Pi 4 and ESP32-S3 ensuring real-time assistance and user privacy without heavy dependency on external cloud services. This design philosophy makes

Auralaid a reliable, high-performance, and empowering tool, providing a safety net that fosters autonomy for its users in real-world environments.

2.1 Product Overview

Auralaid is an AI-powered, modular bionic wearable ecosystem specifically engineered for individuals with hearing impairments. It integrates advanced digital signal processing (DSP), speech-to-text (STT) conversion, and medical-grade biometric surveillance into a cohesive, low-cost system. The device operates in tandem with a high-contrast Flutter mobile application that provides real-time captions, emotional feedback, and critical safety notifications.

The primary purpose of the system is to enhance communication clarity, improve situational awareness, and increase independence for users, particularly in noisy or complex environments where traditional hearing aids often fail. By combining edge-computing via the Raspberry Pi 4, IoT connectivity through the ESP32-S3, and localized AI algorithms, the system ensures a holistic hearing-assistive experience that transcends simple sound amplification.

Key Highlights:

- Affordable and Accessible: Developed using cost-effective, open-source hardware including the ESP32-S3 microcontroller and INMP441 MEMS microphones to ensure accessibility in developing regions.
- Intelligent Visualization: Provides real-time, low-latency captions for English and Urdu alongside emotion detection on a "Gen-Z Cyber-Dark" mobile or desktop interface.
- Features environmental noise-level alerts and a Guardian Mode for fall detection, providing immediate user feedback via a micro vibration motor or mobile SOS notifications.
- Includes high-precision tracking of BPM, SpO2, and body temperature, rendered as smoothed ECG waveforms for comprehensive health surveillance.
- Designed with a micro-threaded software pattern that supports future expansion for research, educational, or commercial healthcare applications.

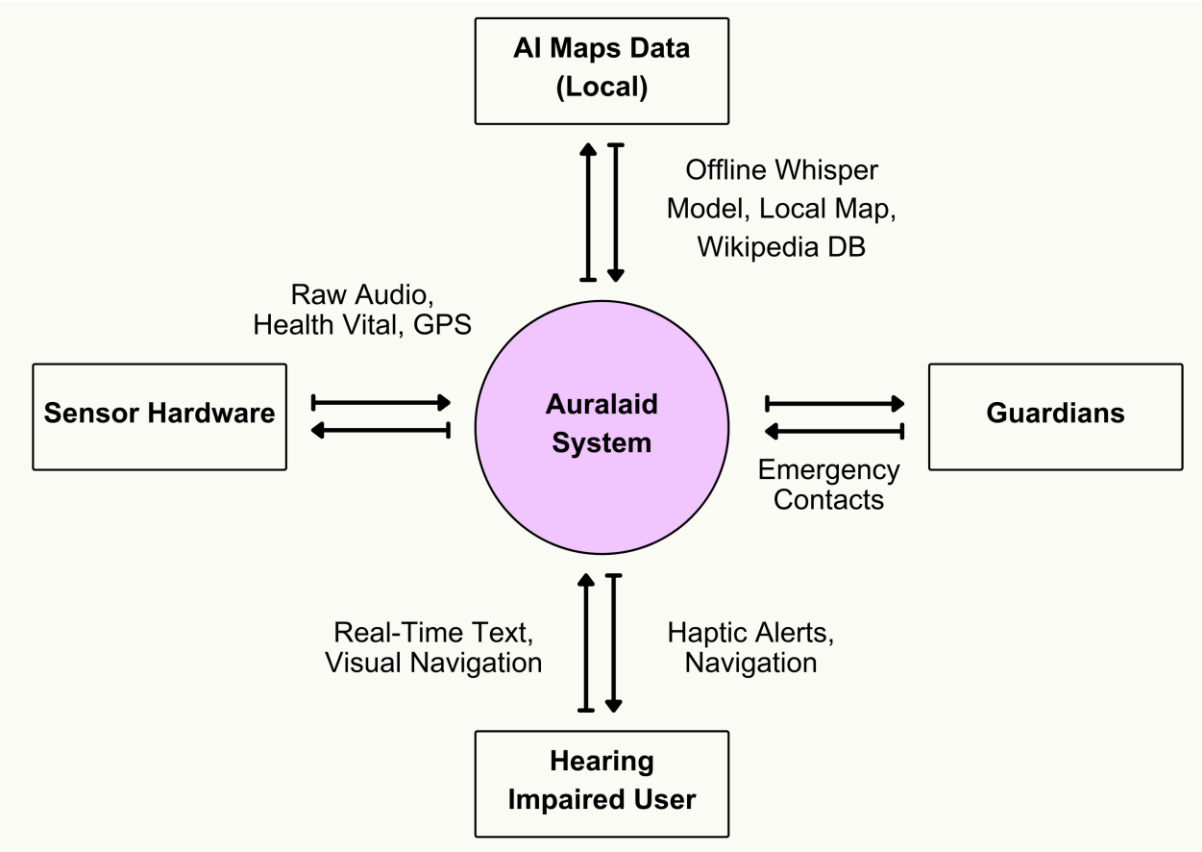


Figure 1: Context Diagram of AuralAid

2.2 Product Functionality

The Auralaid system is engineered to deliver a cohesive bionic experience through several integrated core functions that operate simultaneously across the embedded hardware and mobile interface.

The main functions of the system include:

- **Live Audio Capture:** The system continuously collects environmental acoustic data using high-fidelity INMP441 MEMS microphones, capturing audio at a 16kHz PCM rate for high-accuracy processing.
- **Adaptive Noise Filtering:** Utilizing dual-core hardware and spectral subtraction algorithms, the system reduces background hum and interference to significantly enhance speech clarity for the user.
- **Speech-to-Text Conversion:** Spoken words are processed via the OpenAI Whisper engine on the Raspberry Pi 4 and converted into readable captions displayed on the mobile app in real-time.

- **Emotion Detection:** The system analyzes vocal tone, pitch, and intensity to identify emotional states such as happiness, sadness, or stress providing the user with critical social context.
- **Environmental Noise Alerts:** The device monitors decibel levels and triggers micro vibration motor feedback or visual "Smart Sound Radar" alerts when noise exceeds safe thresholds.
- **Medical-Grade Health Monitoring:** It continuously tracks vital signs, including heart rate (BPM), blood oxygen levels (SpO_2), and body temperature, utilizing the MAX30101 and TMP117 sensors.
- **Intelligent Safety Protocols:** Through Guardian Mode, the system utilizes accelerometer data to detect falls (3.5G impact) and initiates SOS protocols with location sharing.
- **Cross-Platform Connectivity:** The system ensures seamless synchronization of transcripts, biometric telemetry, and safety alerts between the Raspberry Pi BLE Server and the Flutter mobile application.

2.3 Design and Implementation Constraints

The development of the Auralaid ecosystem is governed by several technical and practical boundaries that influence the system architecture and component selection. These constraints ensure the device remains functional as a bionic assistant while operating within the limits of portable hardware.

While developing Aural Aid, several technical and practical constraints need to be considered:

- **Hardware Limitations:** Processing power and memory constraints of microcontrollers (ESP32-S3) limit the complexity of AI models deployed locally.
- **Battery Life:** Continuous audio processing, AI computations, and wireless connectivity can drain the battery quickly, requiring optimized power management.
- **Real-Time Performance:** The system must maintain low-latency processing to display captions and alerts almost instantaneously.
- **Environment Dependence:** Extreme background noise may affect speech recognition and emotion detection accuracy.
- **User Comfort:** The wearable device should remain lightweight, ergonomic, and unobtrusive for extended use.

- **Software Dependencies:** AI models, libraries, and APIs must be compatible with embedded systems and mobile applications.

2.4 Assumptions and Dependencies

The successful deployment and continuous operation of the Auralaid system rely on several critical internal and external factors. These assumptions and dependencies define the baseline conditions required for the bionic assistant to function as a medical-grade companion.

- **Hardware Integrity and Availability:** It is assumed that high-fidelity components including the INMP441 MEMS microphones, ESP32-S3 microcontrollers, and the MAX30101 biometric sensors—are fully functional and correctly integrated into the custom PCB.
- **Mobile Interface Compatibility:** The system assumes that users possess a smartphone capable of running the Flutter framework to interface with the wearable via the high-contrast "OLED-black" GUI.
- **Stable Wireless Connectivity:** Real-time data synchronization between the Raspberry Pi 4 BLE Server and the mobile application depends on a consistent Bluetooth or Wi-Fi connection.
- **AI Model Accuracy and Training:** The OpenAI Whisper and emotion detection models are assumed to be sufficiently trained to handle diverse voices, accents (specifically English and Urdu), and varying levels of background noise.
- **User Compliance and Proper Wear:** Optimal performance of biometric monitoring (BPM, SpO2) and fall detection requires that users wear the device correctly against the skin and follow standard operating guidelines.
- **Availability of Open-Source Frameworks:** The project depends on the continued accessibility of critical open-source libraries and platforms, including Python 3.11, faster-whisper, PyQt5, and SQLite3.
- **Power and Environmental Conditions:** It is assumed that a stable 5V/3A power supply is available for the Raspberry Pi 4 and that the user has occasional Wi-Fi access for Google Maps and Wikipedia API queries.

3. Specific Requirements

This section defines the hardware and software interfaces that allow users to interact with the Auralaid system. These interfaces are engineered to facilitate a seamless transition between the Wearable PCB Unit and the Command Hub (Raspberry Pi 4) while maintaining the high-contrast OLED-black aesthetic required for visual clarity. The system provides tailored dashboards for real-time transcription, medical-grade biometric evaluation, and safe navigation protocols.

3.1 External Interface Requirements

This section describes the system requirements for the external interfaces of the **Auralaid** platform. It details how users will interact with the system through specialized interfaces to access secure, bionic-assisted features. The features include a centralized health and communication dashboard, real-time speech-to-text presentation, and high-fidelity biometric evaluations. These interfaces are designed to be touch-sensitive, easy to navigate, and adaptive to both the 7-inch embedded display and mobile devices.

3.1.1 User Interfaces

The user interaction is driven by a dual-platform approach a PyQt5-based desktop hub and a Flutter-based mobile application ensuring powerful functionality independent of the user's primary device. Users will interact with the platform through:

1. **System Dashboard:** Upon booting, users access a personalized dashboard on the Pi touchscreen or mobile app to view real-time health evaluations, including heart rate, SpO2, and body temperature.
2. **Live Bionic Transcription Module:** A dedicated interface where users can view continuous speech-to-text captions from OpenAI Whisper, presented in a high-contrast format for maximum readability.
3. **Bio-Vitals Monitor:** An interface that renders live, mathematically smoothed ECG waveforms and health logs, providing professional-grade visual feedback of the user's physical condition.
4. **Smart Safety Map**: A GIS-based interface that overlays "Silent Dark" themed Google Maps with real-time GPS breadcrumbs and color-coded noise zones.
5. **AI Neural Assistant (Chat):** A conversational interface powered by Qwen/Wikipedia where users can query contextual information or seek knowledge assistance.

6. **Navigation System:** The UI includes a persistent bottom navigation bar (DASH, LIVE, CHAT, MAP, SYS) for rapid access to all critical bionic and safety features.

3.1.2 Hardware Interfaces

As a modular embedded system, Auralaid relies on specific hardware-level integrations to bridge the physical and digital environments:

1. **Sensor Hub (I2C):** The system interfaces directly with the MAX30101, TMP117, and MLX90614 sensors via the I2C protocol for high-precision biometric data acquisition.
2. **Digital Audio (I2S):** A hardware interface for the INMP441 MEMS microphone using the I2S protocol to ensure low-noise, high-fidelity audio capture for AI inference.
3. **Haptic Feedback (GPIO):** A physical interface to a micro vibration motor controlled via PWM on GPIO Pin 17 to provide tactile alerts for sirens or falls.
4. **Satellite Positioning (UART):** The Neo 8m GPS module interfaces through UART serial pins to provide high-sensitivity location data for the safety map.
5. **Compute & Communication Nodes:** The Raspberry Pi 4 (AI Node) and ESP32-S3 (Sensor Node) interact via UART and BLE to synchronize data across the bionic ecosystem.
6. **Hardware Design & PCB Schematics:** This section presents the PCB schematic design of the proposed system. The schematic shows power management, sensors, microcontroller interfacing, and communication components used in the system.

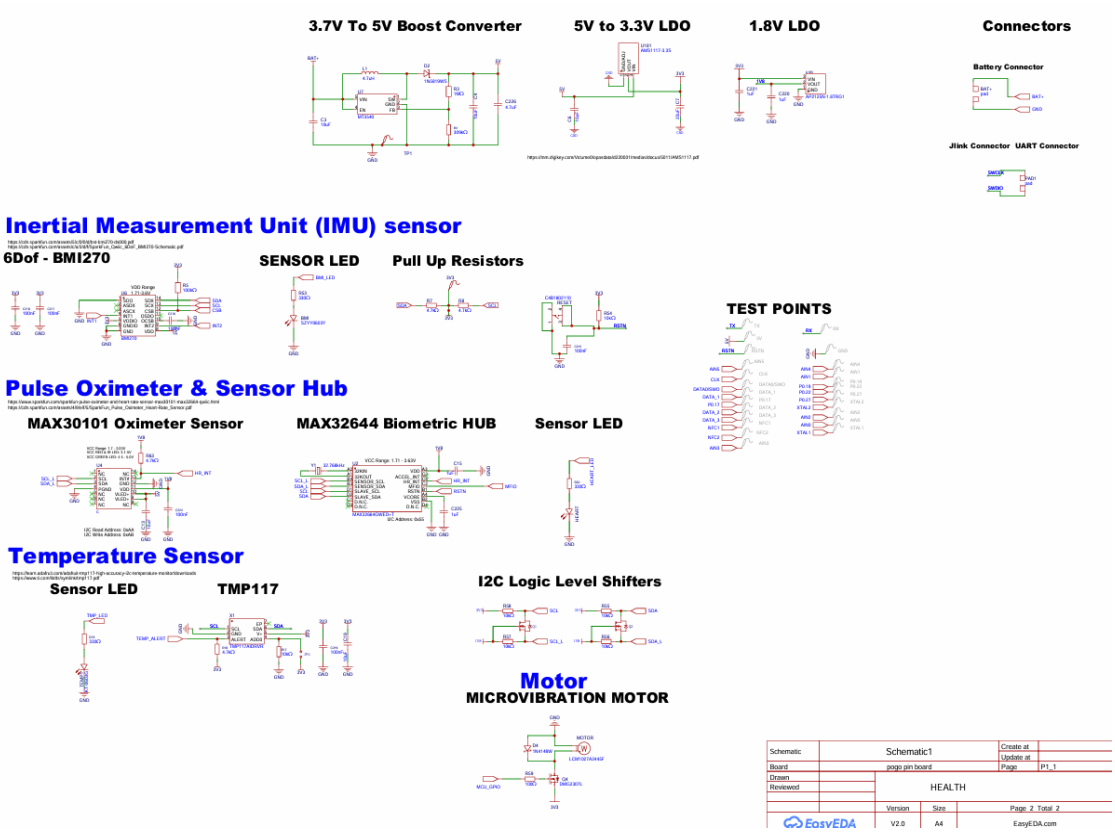


Figure 2: PCB Schematics of Auralaid

3.1.3 Software Interfaces

The Auralaid platform integrates several software components to ensure smooth, real-time bionic functionality:

1. **BLE Server Architecture:** A custom Bluetooth Low Energy interface that broadcasts telemetry data from the Raspberry Pi to the Flutter mobile application.

2. **AI Inference Engines:** Integration with faster-whisper and Qwen AI models via CTranslate2 to provide instantaneous speech processing and knowledge assistance**.**

3. **Database Interface:** The backend interacts with a local SQLite3 database to persistently store health logs, GPS history, and transcription sessions.

4. **Mapping & GIS APIs**: Integration with the Google Maps API and QtWebEngine to render safe-route navigation and environmental noise overlays**.**

5. **Safety SOS Protocols:** A software layer that triggers haptic feedback and automated SOS messaging when the accelerometer detects a 3.5G impact.

3.2 Functional Requirements

The following section details the primary capabilities of the Auralaid ecosystem. It outlines the core tasks users will perform to manage their bionic experience, ranging from real-time transcription and biometric monitoring to emergency safety protocols. These functionalities are essential for delivering a high-quality assistive platform that meets the rigorous expectations of individuals with hearing impairments.

3.2.1 User & System Profile Management

a) Description and Priority:

Enable users to configure their bionic preferences and manage medical-grade profiles, ensuring a personalized assistive experience. This is a high-priority feature for establishing user-specific health baselines.

b) Stimulus/Response Sequence:

Users input personal health data and communication preferences; the system instantly saves these configurations to a local configuration file (e.g., .json or .ini) and reflects them in the bionic dashboard.

c) Functional Requirements:

1. **FR1:** The system shall allow users to create a local profile including basic details and specific hearing impairment levels.
2. **FR2:** The system shall allow users to edit emergency contact information for the Guardian Mode SOS protocol, stored locally in the application memory.
3. **FR3:** The system shall enable users to set preferred languages for transcription (e.g., English or Urdu).
4. **FR4:** The system shall store all user configurations in local file storage and reflect changes instantly across the desktop and mobile interfaces.
5. **FR5:** The system shall allow users to configure haptic alert thresholds for biometric vitals and environmental noise.
6. **FR6:** The system shall provide privacy options to control the temporary logging of biometric and transcript data to local files.

3.2.2 Real-Time Bionic Transcription Process

a) Description and Priority:

Facilitate continuous speech-to-text conversion to bridge communication gaps, which is the core high-priority functionality of the system.

b) Stimulus/Response Sequence

The INMP441 microphone captures audio; the system applies Adaptive Noise Cancellation and Whisper AI inference, presenting live captions on the dashboard.

c) Functional Requirements

1. **FR7:** The system shall capture 16kHz PCM audio and perform real-time spectral subtraction to reduce background noise.
2. **FR8:** The system shall present live transcriptions in a high-contrast, large-font visual format.
3. **FR9**: The system shall support multi-language transcription specifically for English and Urdu scripts.
4. **FR10**: The system shall provide an AI Neural Assistant to analyze transcriptions and offer contextual knowledge or summaries.
5. **FR11:** The system shall allow users to view the current session's transcription history in real-time without requiring a database query.

3.2.3 Biometric Safety Monitoring

a) Description and Priority:

Allow for continuous, medical-grade surveillance of user vitals to ensure physical well-being. This is a high-priority safety feature

b) Stimulus/Response Sequence:

The MAX30101 and TMP117 sensors poll data; the system renders smoothed ECG-style waveforms and triggers haptic pulses if vitals fall outside safe ranges.

c) Functional Requirements:

1. **FR12:** The system shall poll heart rate, SpO2, and body temperature at a minimum frequency of 1Hz.
2. **FR13:** The system shall apply mathematical smoothing algorithms to render fluid biometric waveforms on the mobile interface.
3. **FR14:** The system shall trigger a micro vibration motor alert if SpO2 falls below 92% or heart rate exceeds user-defined limits.
4. **FR15:** The system shall stream biometric data directly to the GUI for real-time visualization without intermediate database storage.

3.2.4 Safe Navigation & Guardian Mode

a) Description and Priority:

Protect users in outdoor environments through impact detection and noise-aware routing. This is a high-priority safety module.

b) Stimulus/Response Sequence:

The accelerometer detects a high-G impact; the system initiates a countdown and, if not cancelled, broadcasts the GPS location to emergency contacts.

c) Functional Requirements:

1. **FR16:** The system shall utilize the device accelerometer to detect falls based on a 3.5G impact threshold.
2. **FR17:** The system shall overlay "Silent Dark" themed maps with color-coded safe (quiet) vs. noisy zones.
3. **FR18:** The system shall fetch real-time location data via the Neo 8m GPS module and display a precision breadcrumb tracker.
4. **FR19:** The system shall trigger an SOS protocol including location sharing and persistent haptic feedback upon unacknowledged fall detection**.**

3.2.5 Centralized Bionic Dashboard

a) Description and Priority:

Provide a unified interface for tracking all real-time activities and environmental alerts. This is a high-priority engagement feature.

b) Stimulus/Response Sequence:

Users access the dashboard to see an overview of transcription sessions, current vitals, and map-based safety logs.

c) Functional Requirements:

1. **FR20:** The system shall provide a dashboard displaying a summary of the current session's transcription accuracy and biometric stability.
2. **FR21**: The system shall allow users to track real-time health metrics and current noise exposure levels.
3. **FR22:** The system shall enable rapid switching between the live audio stream, health waveforms, and the safety map.
4. **FR23:** The system shall dynamically tailor the dashboard layout according to whether the user is in "Stationary" (Desktop Hub) or "Mobile" mode.

3.3 Use Case Model

The Use Case Model provides an organized flow of how users engage with the Auralaid system's core bionic and safety features. These use cases describe the interaction between the primary actor (the user) and the modular embedded system to ensure a seamless assistive experience:

3.3.1 Use Case #1: Power On Device:

Table 3: Use Case of Power on Device

Field	Details
Use Case ID	UC-001
Use Case Name	Power On Device
Business Objectives	**BO-001:** Ensure reliable startup of the hearing aid device.**BO-002:** Initialize all sensors and firmware safely.**BO-003:** Provide immediate feedback that the device is operational.
Dependencies	Battery charged. Firmware correctly flashed. Power button hardware functional.
Description	Allows the user to power on the smart hearing aid device and initialize internal components including ESP32-S3, sensors, and communication modules.
Preconditions	1. Battery is inserted and charged.2. Device is in powered-off state.
Primary Actor	User
Ordinary Sequence	UC-1.1 User presses power button. UC-1.2 System supplies power to MCU.UC-1.3 Firmware initializes sensors and memory. UC-1.4 System enters idle/ready mode.
Postconditions	Device is powered ON and ready for operation.
Expectations	Startup should complete within a few seconds with LED/vibration indication.
Comments	This use case is critical for all other system operations.

3.3.2 Use Case #2: Pair Device with Mobile App

Table 4: Use Case of Pair Device with Mobile App

Field	Details
Use Case ID	UC-002
Use Case Name	Pair Device with Mobile App
Business Objectives	**BO-001:** Enable wireless communication.**BO-002:** Allow real-time data transfer.**BO-003:** Ensure secure device pairing.
Dependencies	Bluetooth/Wi-Fi enabled. Mobile app installed.
Description	Enables pairing between the smart hearing aid and the mobile application using BLE or Wi-Fi.
Preconditions	1. Device is powered ON.2. Mobile app is launched.
Primary Actor	User
Ordinary Sequence	UC-2.1 User opens pairing screen. UC-2.2 App scans for nearby devices. UC-2.3 User selects device. UC-2.4 Secure connection established.
Postconditions	Device and app are successfully paired.
Expectations	Pairing must be secure and user-friendly.
Comments	Required for captions, health data, and alerts.

3.3.3 Use Case #3: User Registration

Table 5: Detailed Use Case of User Registration

Field	Details
Use Case ID	UC-003
Use Case Name	User Registration
Business Objectives	**BO-001:** Create unique user identity.**BO-002:** Store personalized settings.**BO-003:** Enable health data tracking.
Dependencies	Mobile app UI. Local or cloud storage.
Description	Allows new users to register by providing basic profile information.
Preconditions	Mobile app installed.
Primary Actor	User

Ordinary Sequence	UC-3.1 User opens registration screen. UC-3.2 User enters details. UC-3.3 System validates input. UC-3.4 Profile is created.
Postconditions	User account is successfully created.
Expectations	Registration should be simple and minimal.
Comments	Required for personalized experience.

3.3.4. Use Case #4: Sync Device Data

Table 6: Use Case of Sync Device Data

Field	Details
Use Case ID	UC-005
Use Case Name	Sync Device Data
Business Objectives	**BO-001:** Keep data consistent.**BO-002:** Enable real-time monitoring.
Dependencies	Active device-app connection.
Description	Synchronizes audio, captions, and health data from device to mobile app.
Preconditions	Device paired with app.
Primary Actor	Smart Device
Ordinary Sequence	UC-5.1 Device collects data. UC-5.2 Data transmitted wirelessly. UC-5.3 App stores and displays data.
Postconditions	Latest data available on app.
Expectations	Sync must be reliable and low-latency.
Comments	Core functionality of the system.

3.3.4 Use Case #5: Capture Live Audio

Table 7: Use Case of Capture Live Audio

Field	Details
Use Case ID	UC-006
Use Case Name	Capture Live Audio

Business Objectives	**BO-001:** Capture environmental sound in real time using MEMS microphone.**BO-002:** Provide raw audio input for DSP and AI processing.**BO-003:** Ensure continuous and low-latency audio acquisition.
Dependencies	MEMS microphone operational. Audio driver initialized. Device powered ON.
Description	The smart hearing aid captures live environmental audio through the MEMS microphone for further processing.
Preconditions	1. Device is powered ON.2. Audio subsystem initialized successfully.
Primary Actor	Smart Hearing Aid Device
Ordinary Sequence	UC-6.1 System activates MEMS microphone. UC-6.2 Audio signals are continuously sampled. UC-6.3 Raw audio stream is forwarded to DSP module.
Postconditions	Live audio stream is available for processing.
Expectations	Audio capture should be continuous with minimal noise and delay.
Comments	This use case is the foundation for speech enhancement and captioning.

3.3.5 Use Case #6: Filter Background Noise

Table 8: Detailed Use Case of Filter Background Noise

Field	Details
Use Case ID	UC-007
Use Case Name	Filter Background Noise
Business Objectives	**BO-001:** Improve speech clarity.**BO-002:** Reduce ambient noise interference.**BO-003:** Enhance listening comfort for the user.
Dependencies	DSP algorithms available. Live audio input from UC-006.
Description	The system applies digital signal processing techniques to suppress background noise and isolate speech signals.
Preconditions	Live audio is being captured.
Primary Actor	Smart Hearing Aid Device

Ordinary Sequence	UC-7.1 DSP module receives raw audio. UC-7.2 Noise suppression algorithms are applied. UC-7.3 Cleaned audio is forwarded for amplification.
Postconditions	Background noise is reduced in the processed audio stream.
Expectations	Noise reduction must not distort speech signals.
Comments	Essential for usability in noisy environments.

3.3.7 Use Case #7: Amplify Speech Signal

Table 9: Detailed Use Case of Amplify Speech Signal

Field	Details
Use Case ID	UC-008
Use Case Name	Amplify Speech Signal
Business Objectives	**BO-001:** Enhance speech audibility.**BO-002:** Compensate for hearing loss levels.**BO-003:** Provide adjustable amplification.
Dependencies	Clean audio from UC-007.User-defined amplification settings.
Description	The system amplifies processed speech signals to suitable levels for hearing-impaired users.
Preconditions	Background noise filtering completed.
Primary Actor	Smart Hearing Aid Device
Ordinary Sequence	UC-8.1 System receives cleaned audio. UC-8.2 Gain control is applied based on user profile. UC-8.3 Amplified signal is sent to speaker output.
Postconditions	User hears amplified and clearer speech.
Expectations	Amplification must be comfortable and distortion-free.
Comments	Core hearing assistance functionality.

3.3.8 Use Case #8: Monitor Environmental Noise Level

Table 10: Use Case of Monitor Environmental Noise Level

Field	Details
Use Case ID	UC-009
Use Case Name	Monitor Environmental Noise Level

Business Objectives	**BO-001:** Track ambient noise exposure.**BO-002:** Prevent hearing damage.**BO-003:** Support safety alerts.
Dependencies	Audio input stream. dB level calculation module.
Description	The system continuously monitors environmental sound intensity and calculates noise levels in decibels (dB).
Preconditions	Device is active and capturing audio.
Primary Actor	Smart Hearing Aid Device
Ordinary Sequence	UC-9.1 System samples environmental audio. UC-9.2 dB level is calculated. UC-9.3 Noise level is compared with safety thresholds.
Postconditions	Current environmental noise level is determined.
Expectations	Noise level readings must be accurate and real-time.
Comments	Supports proactive hearing protection.

3.3.9 Use Case #9: Trigger Noise Alert

Table 11: Use Case of Trigger Noise Alert

Field	Details
Use Case ID	UC-010
Use Case Name	Trigger Noise Alert
Business Objectives	**BO-001:** Warn users about harmful noise levels.**BO-002:** Encourage safer environments.**BO-003:** Protect long-term hearing health.
Dependencies	Noise level data from UC-009.Vibration motor / alert module.
Description	The system alerts the user when environmental noise exceeds predefined safe thresholds.
Preconditions	Noise monitoring is active.
Primary Actor	Smart Hearing Aid Device
Ordinary Sequence	UC-10.1 System detects excessive noise level. UC-10.2 Alert condition is triggered. UC-10.3 Device vibrates or sends notification to app.
Postconditions	User is notified of unsafe noise exposure.
Expectations	Alerts must be noticeable but non-disruptive.
Comments	Improves user safety in loud environments.

3.3.10 Use Case #10: Convert Speech to Text

Table 12: Use Case of Convert Speech to text

Field	Details
Use Case ID	UC-011
Use Case Name	Convert Speech to Text
Business Objectives	**BO-001:** Enable real-time speech understanding for hearing-impaired users.**BO-002:** Provide accurate textual representation of spoken language.**BO-003:** Support communication in noisy or multi-speaker environments.
Dependencies	Clean audio input from UC-008.Speech-to-Text AI model available. Stable device–app connection.
Description	The system converts amplified speech signals into readable text using AI-based speech recognition.
Preconditions	1. Live audio is being captured and processed.2. Device is connected to the mobile application.
Primary Actor	Smart Hearing Aid Device
Ordinary Sequence	UC-11.1 Processed audio is forwarded to STT module. UC-11.2 AI model analyzes speech patterns. UC-11.3 Recognized text is generated. UC-11.4 Text is sent to mobile app.
Postconditions	Spoken speech is successfully converted into text.
Expectations	Text conversion should be near real-time with acceptable accuracy.
Comments	This use case forms the core accessibility feature of the system.

3.3.11 Use Case #11: Display Live Captions

Table 13: Use Case of Display Live Captions

Field	Details
Use Case ID	UC-012
Use Case Name	Display Live Captions
Business Objectives	**BO-001:** Provide visual communication support.**BO-002:** Improve conversation comprehension.**BO-003:** Enhance inclusivity for hearing-impaired users.

Dependencies	Text output from UC-011.Mobile application UI module.
Description	The mobile application displays real-time captions generated from spoken speech.
Preconditions	Speech-to-text conversion is active.
Primary Actor	Mobile Application
Ordinary Sequence	UC-12.1 App receives text stream. UC-12.2 Text is formatted for readability. UC-12.3 Captions are displayed on screen.
Postconditions	User can read live captions in real time.
Expectations	Captions must be clear, high-contrast, and easy to read.
Comments	Essential for day-to-day communication.

3.3.12 Use Case #12: Detect Speaker Emotion

Table 14: Use Case of Detect speaker Emotions

Field	Details
Use Case ID	UC-013
Use Case Name	Detect Speaker Emotion
Business Objectives	**BO-001:** Add contextual understanding to conversations.**BO-002:** Improve social interaction awareness.**BO-003:** Assist users in interpreting emotional tone.
Dependencies	Processed voice data. Emotion detection AI model.
Description	The system analyzes voice characteristics to detect the emotional state of the speaker.
Preconditions	Speech audio is available for analysis.
Primary Actor	AI Engine
Ordinary Sequence	UC-13.1 Audio features are extracted. UC-13.2 AI model evaluates emotional cues. UC-13.3 Emotion label is generated.
Postconditions	Speaker emotion is identified.
Expectations	Emotion detection should be supportive, not misleading.
Comments	Enhances contextual awareness but is non-critical.

3.3.13 Use Case #13: Display Emotion Indicator

Table 15: Use Case of Display Emotion Indicator

Field	Details
Use Case ID	UC-014
Use Case Name	Display Emotion Indicator
Business Objectives	**BO-001:** Visually represent emotional context.**BO-002:** Improve conversation interpretation.**BO-003:** Reduce social misunderstandings.
Dependencies	Emotion data from UC-013.Mobile app UI components.
Description	The mobile application displays emotion indicators (icons or labels) alongside live captions.
Preconditions	Emotion detection is active.
Primary Actor	Mobile Application
Ordinary Sequence	UC-14.1 App receives emotion label. UC-14.2 Emotion icon is selected. UC-14.3 Indicator is displayed with captions.
Postconditions	Emotion indicator is visible to the user.
Expectations	Indicators should be simple and non-intrusive.
Comments	Optional feature enhancing user experience.

3.3.14 Use Case #14: Summarize Conversation

Table 16: Use Case of summarize Conversation

Field	Details
Use Case ID	UC-015
Use Case Name	Summarize Conversation
Business Objectives	**BO-001:** Help users recall important discussions.**BO-002:** Reduce cognitive load from long conversations.**BO-003:** Provide quick review of spoken interactions.
Dependencies	Stored caption data.AI summarization module.
Description	The system generates a concise summary of a completed conversation using AI-based summarization techniques.
Preconditions	1. A conversation session has ended.2. Caption data is stored.

Primary Actor	AI Engine
Ordinary Sequence	UC-15.1 System gathers caption history. UC-15.2 AI model extracts key points. UC-15.3 Summary is generated and stored.
Postconditions	Conversation summary is available for review.
Expectations	Summary should be brief, clear, and relevant.
Comments	Useful for meetings, lectures, and medical discussions.

3.3.15 Use Case #15: Monitor Heart Rate

Table 17: Use Case of Monitor Heart Rate

Field	Details
Use Case ID	UC-016
Use Case Name	Monitor Heart Rate
Business Objectives	**BO-001:** Continuously track user heart rate.**BO-002:** Support health awareness and early detection of abnormalities.**BO-003:** Provide reliable physiological monitoring.
Dependencies	MAX30101 sensor operational. Sensor driver initialized. Device powered ON.
Description	The smart hearing aid measures the user's heart rate using the integrated MAX30101 sensor.
Preconditions	1. Device is powered ON.2. Heart rate sensor is properly attached and calibrated.
Primary Actor	Smart Hearing Aid Device
Ordinary Sequence	UC-16.1 System activates heart rate sensor. UC-16.2 Sensor captures pulse data. UC-16.3 System calculates heart rate (BPM). UC-16.4 Data is stored/transmitted to app.
Postconditions	Current heart rate value is available.
Expectations	Heart rate readings must be accurate and consistent.
Comments	Useful for monitoring stress and physical activity levels.

3.3.16 Use Case #16: Monitor SpO_2 Level

Table 18: Use Case of Monitor SpO2 level

Field	Details
Use Case ID	UC-017
Use Case Name	Monitor SpO_2 Level
Business Objectives	**BO-001:** Track blood oxygen saturation levels.**BO-002:** Detect potential respiratory issues.**BO-003:** Support overall health monitoring.
Dependencies	MAX30101 sensor. Valid sensor contact with skin.
Description	The system measures the user's blood oxygen saturation (SpO_2) using optical sensing.
Preconditions	1. Device is worn correctly.2. Sensor is active and calibrated.
Primary Actor	Smart Hearing Aid Device
Ordinary Sequence	UC-17.1 Sensor emits light signals. UC-17.2 Reflected signals are captured. UC-17.3 SpO_2 percentage is calculated. UC-17.4 Value is logged and synced.
Postconditions	SpO_2 level is recorded.
Expectations	Measurements should fall within medical tolerance range.
Comments	Helpful for users with respiratory conditions.

3.3.17 Use Case #17: Monitor Body Temperature

Table 19: Use Case of Monitor Body Temperature

Field	Details
Use Case ID	UC-018
Use Case Name	Monitor Body Temperature
Business Objectives	**BO-001:** Track user body temperature.**BO-002:** Detect fever or abnormal temperature trends.**BO-003:** Enhance health safety.
Dependencies	TMP117 temperature sensor. Stable skin contact.
Description	The system measures body temperature using a digital temperature sensor embedded in the device.
Preconditions	Device is worn properly.

Primary Actor	Smart Hearing Aid Device
Ordinary Sequence	UC-18.1 System activates temperature sensor. UC-18.2 Sensor reads temperature values. UC-18.3 Data is converted to °C/°F. UC-18.4 Temperature is stored and synced.
Postconditions	Body temperature data is available.
Expectations	Temperature readings should be stable and precise.
Comments	Supports early detection of illness.

3.3.18 Use Case #18: Detect Abnormal Health Values

Table 20: Use Case of Detect Abnormal Health Values

Field	Details
Use Case ID	UC-019
Use Case Name	Detect Abnormal Health Values
Business Objectives	**BO-001:** Identify health risks early.**BO-002:** Compare vitals against safe thresholds.**BO-003:** Trigger timely alerts.
Dependencies	Health data from UC-016, UC-017, UC-018.Predefined threshold values.
Description	The system analyzes collected health data to detect abnormal heart rate, SpO_2, or temperature values.
Preconditions	Health monitoring is active.
Primary Actor	Smart Hearing Aid Device
Ordinary Sequence	UC-19.1 System retrieves latest health readings. UC-19.2 Values are compared with thresholds. UC-19.3 Abnormal condition is detected.
Postconditions	Abnormal health condition is flagged.
Expectations	Detection must be reliable to avoid false alarms.
Comments	Acts as a preventive safety mechanism.

3.3.19 Use Case #19: Send Health Alert

Table 21: Use Case of Send Health Alert

Field	Details
Use Case ID	UC-020
Use Case Name	Send Health Alert
Business Objectives	**BO-001:** Notify users of health risks.**BO-002:** Enable caregiver awareness.**BO-003:** Support emergency response.
Dependencies	Abnormal condition from UC-019.Device–app connectivity. Notification service.
Description	The system sends alerts to the user and/or caregiver when abnormal health values are detected.
Preconditions	An abnormal health condition has been identified.
Primary Actor	Mobile Application
Ordinary Sequence	UC-20.1 Alert message is generated. UC-20.2 Notification is sent to mobile app. UC-20.3 Caregiver/admin is notified (if enabled).
Postconditions	Health alert is successfully delivered.
Expectations	Alerts must be timely and clearly understandable.
Comments	Critical for user safety and medical awareness.

3.3.20 Use Case #20: View Caption History

Table 22: Use Case of View caption History

Field	Details
Use Case ID	UC-022
Use Case Name	View Caption History
Business Objectives	**BO-001:** Allow review of past conversations.**BO-002:** Assist memory recall.**BO-003:** Provide reference for important discussions.
Dependencies	Stored caption logs. History management module.
Description	Allows users to access previously generated speech-to-text captions.
Preconditions	Caption history exists.
Primary Actor	User

Ordinary Sequence	UC-22.1 User opens caption history. UC-22.2 System retrieves stored captions. UC-22.3 Captions are displayed chronologically.
Postconditions	Caption history is visible.
Expectations	Text must be readable and searchable.
Comments	Useful for meetings, lectures, and medical advice recall.

3.3.21 Use Case #21: Update Firmware

Table 23: Use Case of Update Firmware

Field	Details
Use Case ID	UC-025
Use Case Name	Update Firmware
Business Objectives	**BO-001:** Improve device performance.**BO-002:** Fix bugs and vulnerabilities.**BO-003:** Add new features safely.
Dependencies	Stable power supply. Firmware update file. Admin authorization.
Description	Allows authorized administrators to update the hearing aid device firmware.
Preconditions	Device is connected and idle.
Primary Actor	Admin
Ordinary Sequence	UC-25.1 Admin initiates update process. UC-25.2 Firmware file is validated. UC-25.3 Update is installed. UC-25.4 Device restarts.
Postconditions	Firmware is successfully updated.
Expectations	Update process must be secure and fail-safe.
Comments	Ensures long-term reliability of the system.

3.3.22 Use Case #22: Adjust Audio Profile

Table 24: Use Case of Adjust Audio Profile

Field	Details
Use Case ID	UC-026
Use Case Name	Adjust Audio Profile

Business Objectives	**BO-001:** Allow users to customize hearing preferences.**BO-002:** Improve listening comfort in different environments.**BO-003:** Support personalized hearing assistance.
Dependencies	Device–app connection. User profile settings module.
Description	Enables users to adjust audio profiles such as volume level, frequency balance, and environment modes (indoor, outdoor, noisy).
Preconditions	1. User is logged in.2. Device is connected to the mobile app.
Primary Actor	User
Ordinary Sequence	UC-26.1 User opens audio settings. UC-26.2 User selects preferred profile or adjusts sliders. UC-26.3 System applies changes in real time. UC-26.4 Settings are saved.
Postconditions	Audio profile is updated and applied.
Expectations	Changes should take effect instantly without distortion.
Comments	Improves user comfort and adaptability in various environments.

3.3.23 Use Case #23: Emergency SOS Alert

Table 25: Use Case of Emergency SOS Alert

Field	Details
Use Case ID	UC-027
Use Case Name	Emergency SOS Alert
Business Objectives	**BO-001:** Provide quick emergency assistance.**BO-002:** Enhance user safety in critical situations.**BO-003:** Enable rapid caregiver notification.
Dependencies	Health abnormality detection OR manual trigger. Mobile network / notification service.
Description	Allows the system to send an emergency SOS alert to caregivers or emergency contacts when triggered manually or automatically.
Preconditions	1. Emergency contacts are configured.2. Device or app is active.
Primary Actor	User
Ordinary Sequence	UC-27.1 User presses SOS button OR system detects critical condition. UC-27.2 Alert message is generated. UC-27.3 Location and vitals are attached. UC-27.4 Alert is sent to emergency contacts.

Postconditions	Emergency alert is successfully delivered.
Expectations	Alert must be fast, reliable, and unmistakable.
Comments	Adds a strong safety feature to the smart hearing aid system.

3.3.24 Use Case #24: Battery Status Monitoring & Alert

Table 26: Use Case of Battery Status Monitoring and Alert

Field	Details
Use Case ID	UC-028
Use Case Name	Battery Status Monitoring & Alert
Business Objectives	**BO-001:** Continuously monitor device battery level.**BO-002:** Prevent unexpected device shutdown.**BO-003:** Inform users in advance for timely recharging.
Dependencies	Battery level sensor / Ecopower management module. Device–app connectivity.
Description	The system monitors the battery level of the smart hearing aid and notifies the user when the battery is low or critically low.
Preconditions	1. Device is powered ON.2. Battery monitoring module is active.
Primary Actor	Smart Hearing Aid Device
Ordinary Sequence	UC-28.1 System reads current battery level. UC-28.2 Battery percentage is calculated. UC-28.3 Level is compared with predefined thresholds. UC-28.4 Low/critical battery alert is generated and sent to app.
Postconditions	User is informed about current battery status.
Expectations	Alerts must be timely and clearly visible/audible.
Comments	Critical for uninterrupted operation of the hearing aid device.

4. Other Non-Functional Requirements

The Auralaid bionic assistant is engineered to be a reliable, high-performance edge-computing solution. Unlike standard cloud-based applications, Auralaid prioritizes local inference and low-latency hardware interaction to ensure the safety and privacy of hearing-impaired users. The system creates a professional-grade assistive environment by meeting strict criteria for real-time responsiveness, hardware stability, and data privacy.

4.1 Performance Requirements

These Key Performance Indicators (KPIs) define the technical benchmarks for the Auralaid bionic ecosystem. To ensure user safety and high-fidelity assistance, the system must meet these requirements under standard operating conditions.

4.1.1 Interview Evaluation Speed

Requirement

The system must achieve a Time-To-First-Byte (TTFB) for partial captions and Final Transcript Latency of seconds from the end of a speech utterance.

Rationale

In assistive hearing technology, any delay exceeding 2 seconds disrupts the "natural turn-taking" of human conversation, causing a cognitive disconnect for the user.

Implementation Details

Audio is packetized into 100 ms chunks via the I2S bus to balance processing efficiency with snappiness.

- **Inference Engine:** Optimized faster-whisper models utilize 8-bit integer quantization on the Raspberry Pi 4 to maximize throughput while maintaining 90%+ transcription accuracy.
- **VAD Threshold:** The Voice Activity Detection (VAD) is tuned to a 500 ms silence threshold to distinguish between natural pauses and the end of an interaction.

4.1.2 Biometric Reporting Speed & Accuracy

Requirement

The system must poll physiological sensors at a frequency of 1Hz (1 sample/sec) with a data jitter to maintain a fluid real-time waveform.

Rationale

High-frequency polling is necessary for the Guardian Mode to detect sudden physiological spikes (e.g., tachycardia or hypoxia) that often precede or follow a physical fall.

Implementation Details

Biometric polling threads are assigned a high-priority "Real-Time" status in the Python/Flutter environment to prevent CPU starvation by the AI engine. A 5-point moving average filter is applied to raw PPG signals to eliminate artifacts caused by wearable movement.

4.1.3 GUI Responsiveness & System Fluidity

Requirement

The interface must maintain a consistent 60 FPS (Frames Per Second) refresh rate. Interaction-to-Next-Paint (INP) for all navigation tasks must be <200ms.

Rationale

Visual stability is critical for hearing-impaired users who rely heavily on visual cues; any layout shift (CLS) or blocking time can lead to user disorientation.

Implementation Hardware Acceleration:

Both PyQt5 and Flutter modules utilize GPU-based rendering on the Raspberry Pi and mobile device to offload the CPU from draw calls. Large datasets (like historical transcription logs) utilize Lazy Loading to ensure that the primary dashboard remains interactive during file I/O operations.

4.2 Safety and Security Requirements

These requirements establish the "fail-safe" protocols and data protection standards necessary for Auralaid to operate as a trusted bionic life-support and communication aid.

4.2.1 Edge-Data Privacy and Localized Sovereignty

Requirement

The system must adhere to a Zero-Cloud policy for all raw acoustic and biometric data. Users must have access to a "Session Sanitize" function that purges temporary RAM buffers and local logs upon the toggling of "Privacy Mode".

Rationale

Individuals with hearing loss are vulnerable to "acoustic eavesdropping." Storing transcripts locally ensures that private conversations never traverse the public internet, mitigating the risk of large-scale data breaches.

Implementation Details

All persistent logs (JSON/CSV) are encrypted at rest using AES-256 with a device-unique hardware key.

- **Volatile Processing:** Transcription buffers are stored in RAM (tmpfs) rather than the SD card to ensure data is unrecoverable upon power loss or session termination.
- **UI Control:** A persistent "Privacy Shield" icon on the dashboard provides one-tap deactivation of all logging functions.

4.2.2 Guardian Mode: Fall Detection and SOS Reliability

Requirement

The system must utilize a Tri-Axial Accelerometer (MPU6050) to detect a specific "Fall Signature": a free-fall phase followed by a high-G impact and a subsequent period of immobility.

Rationale

Accidental falls are a high-risk factor for users with sensory impairments. A automated, low-latency SOS protocol can be life-saving if the user is rendered unconscious or unable to reach their mobile device.

Implementation Details

The IMU (Inertial Measurement Unit) is configured with a Hardware Interrupt to wake the CPU immediately upon impact, bypassing standard software polling cycles.

4.2.3 Hardware Safety and Power Stability

Requirement

The system must perform real-time voltage rails monitoring, triggering a Safe Shutdown or "Low Power Mode" if the input voltage drops below 4.8V or the SoC temperature exceeds 75°C.

Rationale

: AI inference (Whisper) and GPS tracking are highly sensitive to voltage fluctuations; undervoltage can cause "Silent Failures" where the AI produces hallucinations or the sensors provide inaccurate medical readings.

Implementation Details

The system utilizes the BCM2711 internal power management unit to poll the vcos status every 500ms.

4.3 Software Quality Attributes

This section identifies the quality attributes that the Auralaid platform must possess to offer an efficient delivery of the services required in enhancing bionic performance, health reliability, and user usability. These attributes make certain that the system not only fulfills the expectations of the hearing-impaired users but also the execution characteristics, which include dependability, edge-efficiency, extensibility, and modularity of the bionic hub.

4.3.1 Reliability

Requirement

Aim for 99.9% availability during active wear sessions, which translates to a Mean Time Between Failures (MTBF) of greater than 500 hours. For a medical-grade assistive device, it is critical that the core transcription and fall-detection services remain functional without interruption.

Implementation Details

Use a micro-threaded software architecture where sensor polling and AI inference operate in isolated threads. Implement automated watchdog timers and recovery protocols for the BLE server and I2C bus to ensure that if a single sensor stalls, the rest of the system remains operational.

4.3.2 Adaptability

Requirement

The system must allow for easy integration with third-party medical and productivity tools, such as Google Maps API for safe routing, Wikipedia API for the AI assistant, and potential future integration with electronic health record (EHR) systems.

Implementation Details

Utilize a modular API-centric design where external services are accessed via standardized RESTful or JSON-based interfaces, allowing the bionic hub to fetch and process external data without altering the core firmware.

4.3.3 Usability

Requirement

Ensure that 80% of new users are able to initiate a live transcription session and view their first heart rate reading independently within two minutes or less of powering on the device.

Implementation Details

U Conduct user testing with individuals within the target demographic. Implement a high-

contrast "OLED-black" theme with large-font accessibility, intuitive icons, and a persistent "Help" module that provides temporary dynamic guidance across the dashboard.

4.3.4 Maintainability

Requirement

Requirement All backend and firmware code must follow specific coding standards, such as PEP 8 for Python. The project requires an 85%-unit test coverage for core AI and safety modules. Once a technical bug is identified, the resolution and patch deployment time should not exceed 48 hours**.**

Implementation Details

Take advantage of version control tools like Git. Implement Continuous Integration (CI) workflows to run automated tests on every code commit, ensuring that updates to the transcription engine do not break health-monitoring functions.

4.3.5 Scalability

Requirement

The architecture must have the ability to handle a 5x increase in sensor data throughput—such as adding Air Quality (MQ-series) or GSR sensors without a considerable decrease in the quality of the real-time transcription service.

Implementation Details

Leverage the multi-drop capability of the I2C bus and the micro-threaded software pattern. By using a "Task Manager" logic on the Raspberry Pi 4, system resources can be dynamically scaled and prioritized for the most critical sensor nodes during high-load scenarios..

4.3.6 Security

Requirement

Security assessments and hardware-level penetration tests should be carried out at least once every 3 months. Any discovered vulnerabilities regarding user data privacy or unauthorized remote access must be patched within 48 hours.

Implementation Details

Use automated security scanning tools to check for vulnerabilities in the Flask/FastAPI backend and BLE broadcasting. Ensure that the organization addresses all noted vulnerabilities within 48 hours to maintain the "Edge-Privacy" integrity of the user's personal bionic data.

5. Design Requirements

The design specifications for Auralaid are mapped to the hardware and software components that make up the necessary infrastructure to support the bionic and health-monitoring goals of the platform. It is required to match the stringent requirements of real-time performance, edge-security, and high-contrast usability demanded by its users. These requirements cover the system architecture, physical appearance of the wearable, and the synergistic relations between the ESP32-S3 sensor nodes and the Raspberry Pi 4 AI hub. The design also ensures a consistent, high-quality user experience across the embedded touchscreen and the Flutter mobile application during daily use.

5.1 High-Level Design

The high-level system design of Auralaid provides a general knowledge of the modular structure of the bionic assistant. It comprises the Whisper-based transcription engine, the biometric analysis layer, and the Guardian Mode safety logic. While these algorithms can run independently—allowing the user to monitor health without transcription—they also work in synchrony to provide a holistic bionic environment. This design leverages edge-AI computational systems to handle core operations locally, identifying the data feed from I2S/I2C sensors and managing communication between the front-end GUI and back-end hardware workers. Big-picture requirements include system extensibility and low-latency performance that guarantee Auralaid responds properly to real-time environmental stimuli.

5.1.1 Block Diagram

The High-Level Block Diagram gives a clear overview of the system, illustrating the main elements of the project and their functional relationships. This diagram is useful for visualizing how the Auralaid platform is structured, showcasing major blocks such as the Audio Processing Unit, Biometric Sensor Array, and the AI Knowledge Hub. It illustrates how digitized audio and health data move through the system—from raw sensor capture to processed visual feedback—without going into the low-level circuit implementation.

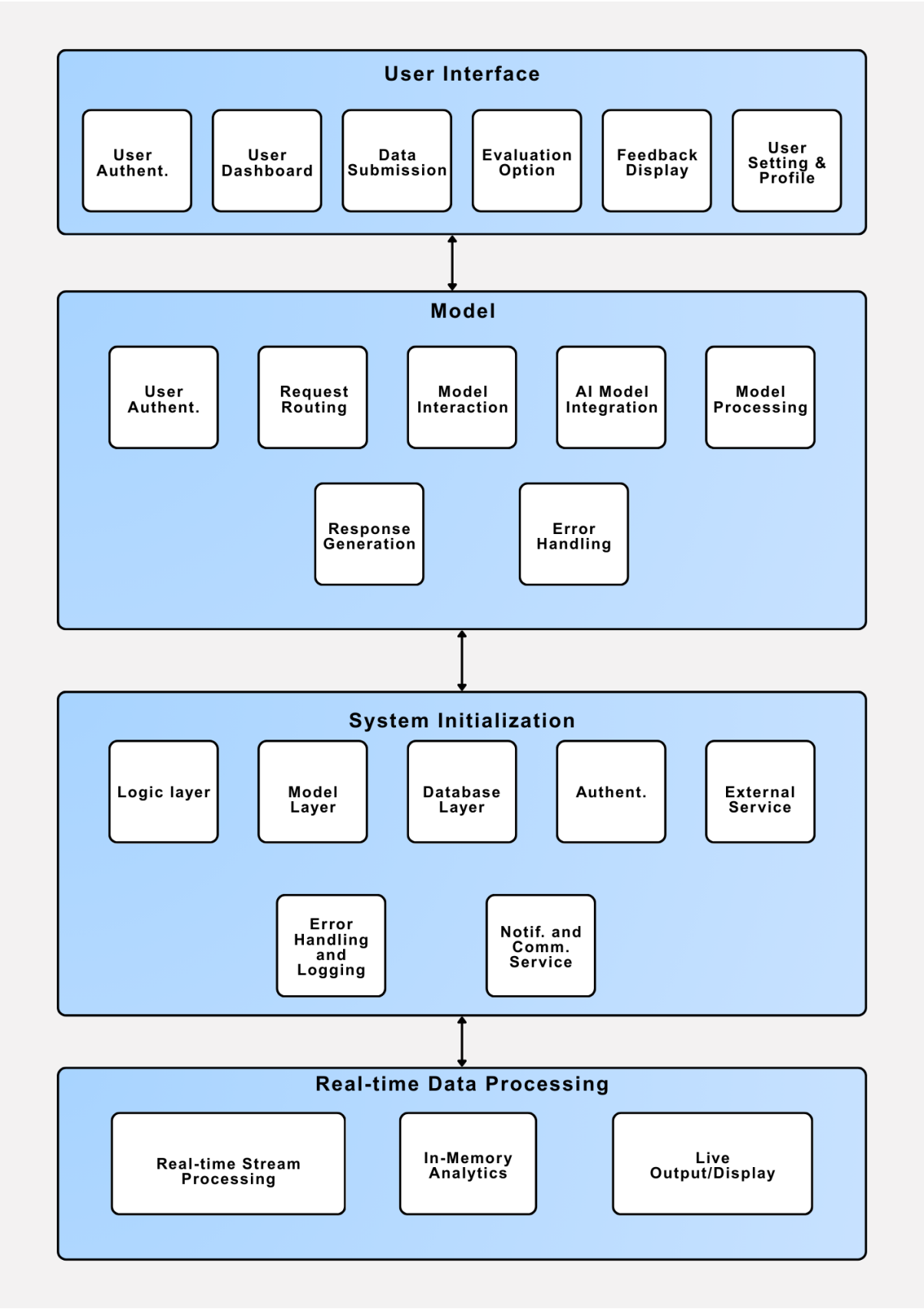


Figure 3: Block Diagram for Auralaid

5.1.2 Design Architecture Patterns

The Design Architecture Patterns section explains the architectural foundation of the Auralaid platform, implemented to ensure the system is scalable, flexible, and maintainable at the edge. The platform utilizes a Client-Server topology where the Raspberry Pi 4 acts as the local "Bionic Hub" (Server) and the 7-inch touchscreen and Flutter app act as "Bionic Clients".

Coding Pattern

Auralaid implements the Strategy Pattern to manage its diverse bionic evaluation processes. This pattern defines a set of encapsulated algorithms (e.g., transcription, biometric analysis, fall detection) and makes them interchangeable at runtime. This allows the system to switch between different languages (English vs. Urdu) or different AI models (Whisper vs. Qwen) without altering the core application logic.

Reason

1. **Dynamic Evaluation Methods**

The bionic services provided by Auralaid require distinct algorithmic approaches. For instance, Real-Time Transcription involves I2S audio processing and Whisper AI, while Health Monitoring requires I2C sensor polling and moving average filters. Using the Strategy Pattern, these methods are defined as separate strategies, allowing the system to toggle between "High-Accuracy Transcription" and "Power-Saving Mode" depending on the user's battery status without affecting health monitoring.

2. **Scalability and Flexibility**

As the bionic assistant evolves, new assessment schemes—such as air quality sensing or gesture recognition—may be integrated. The Strategy Pattern enables the addition of these new evaluation methods without influencing the platform's underlying structure. Whether adding a new dialect for speech recognition or a different biometric sensor, the system can expand its bionic reach with minimal effort.

3. **Separation of Concerns:**

The Strategy Pattern effectively decouples the core bionic logic from the overall system framework. Each evaluation type (transcription, vitals, safety map) is concentrated into separate strategy classes. This modularity simplifies code maintenance and testing, as the biometric logic can be updated or debugged independently from the speech-to-text engine.

4. **Ease of Maintenance:**

Because each evaluation strategy is represented by an independent class, enhancements to an existing method—such as improving the accuracy of the Guardian Mode fall detection—can be performed in isolation. This ensures that a software update to the AI Knowledge Hub does not inadvertently disable the critical safety interrupts of the wearable device.

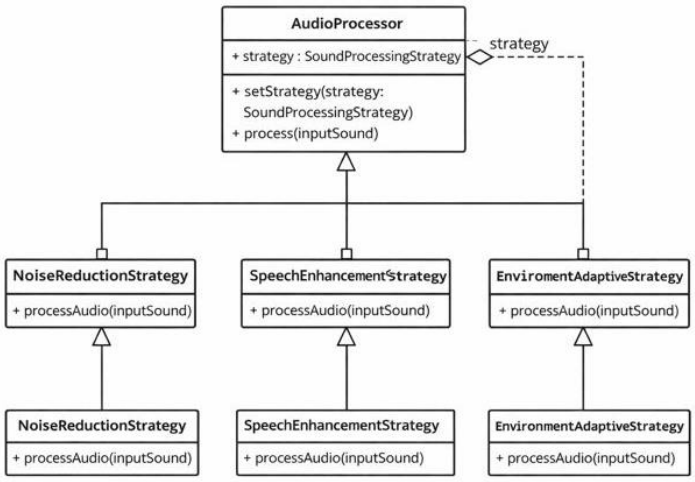


Figure 4: Strategy Pattern for Auralaid

5.2 Structural Design

The Structural Design phase is the critical architectural stage where the bionic components and software modules of the Auralaid platform are defined in detail. This phase focuses on decomposing the complex bionic assistant into manageable subsystems, assigning clear responsibilities to each, and defining the interfaces through which these subsystems interact. By establishing a robust structural framework, Auralaid ensures the high-performance integration of hardware sensors with edge-AI processing.

5.2.1 Class Diagram

This diagram not only depicts the static architecture but also highlights the functional interdependence required for real-time assistance:

Manages personal profiles, localized settings, and emergency contacts.

Specialized classes like TranscriptionEngine and HealthMonitor inherit from a base BionicFeature class, utilizing the Strategy Pattern for flexible algorithmic execution.

Data Models: Entities that define the structure of the local telemetry logs (vitals, transcripts, and GPS breadcrumbs) used for session analysis.

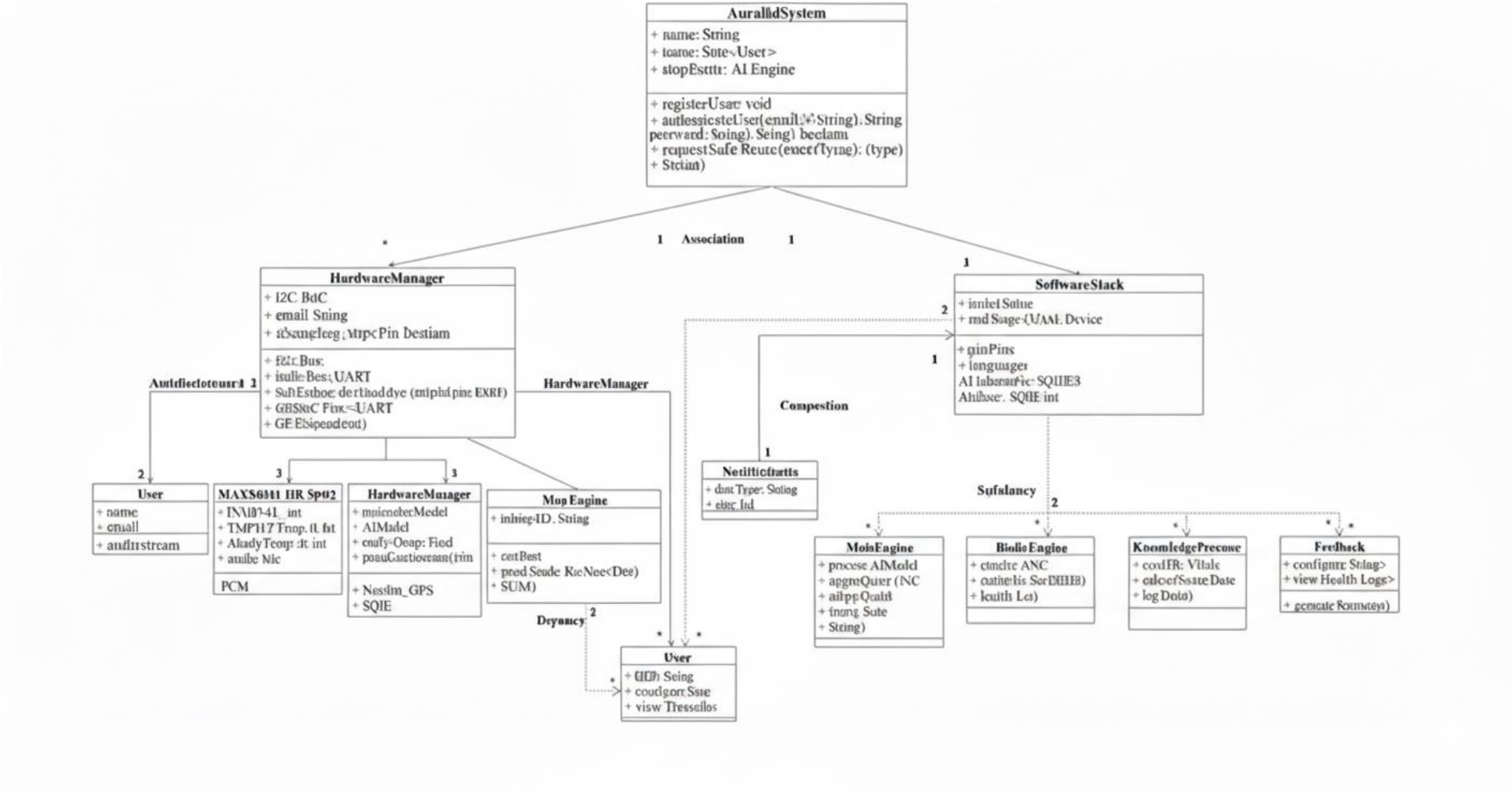


Figure 2: Class Diagram of Auralaid

5.2.2 Component Diagram

The Component Diagram illustrates the structural layout of the Auralaid ecosystem, detailing the physical and logical modules and their specific roles. It reveals the "Edge-to-Mobile" architecture, showing how the front-end interfaces, the AI back-end hub, and the sensory hardware interact. This diagram is essential for maintaining modularity, allowing individual components like the transcription engine or health monitor—to be updated independently i.e., show in **Error! Reference source not found.**

Bionic Authentication & Device Pairing

This component manages secure access to the bionic hub and ensures that the wearable unit is correctly paired with the user's mobile device. It handles user registration and local profile encryption to protect sensitive bionic data. Additionally, it manages token validation for API-based services (like Maps or AI Knowledge) to ensure that only authorized bionic sessions are active.

Transcription & Emotional Analysis

As the core communication module, this component captures raw I2S audio from the INMP441 MEMS microphone. It processes the audio through the Whisper AI inference engine to generate live captions and performs vocal sentiment analysis to provide the user with emotional context of their environment.

Bionic Telemetry Storage (Local Files)

Since the system does not use a central database, this component manages the Local File Storage on the Raspberry Pi's SD card. It is responsible for logging transcription sessions, biometric trends (BPM/SpO2), and GPS breadcrumbs into structured. Json or .csv files for later review in the Results section.

Health & Safety Monitoring

This component aggregates real-time data from the MAX30101, TMP117, and MPU6050 sensors via the I2C bus. It executes the Guardian Mode logic, monitoring for physiological anomalies and physical falls and triggers the haptic feedback or SOS protocols when safety thresholds are breached.

Bionic API Gateway

The API Gateway serves as the communication hub between the embedded PyQt5 front-end and the various back-end AI services and hardware workers. It routes requests for AI-driven knowledge (Qwen/Wikipedia) and manages the BLE Server architecture to ensure seamless data streaming to the Flutter mobile application.

Notification & Alert Service

This service ensures the user is aware of critical system and environmental updates. It manages the delivery of "Smart Sound Radar" alerts for loud noises, biometric warnings for low SpO2, and system-level notifications such as low battery or hardware disconnection, enhancing overall user safety and engagement. show in **Error! Reference source not found.**

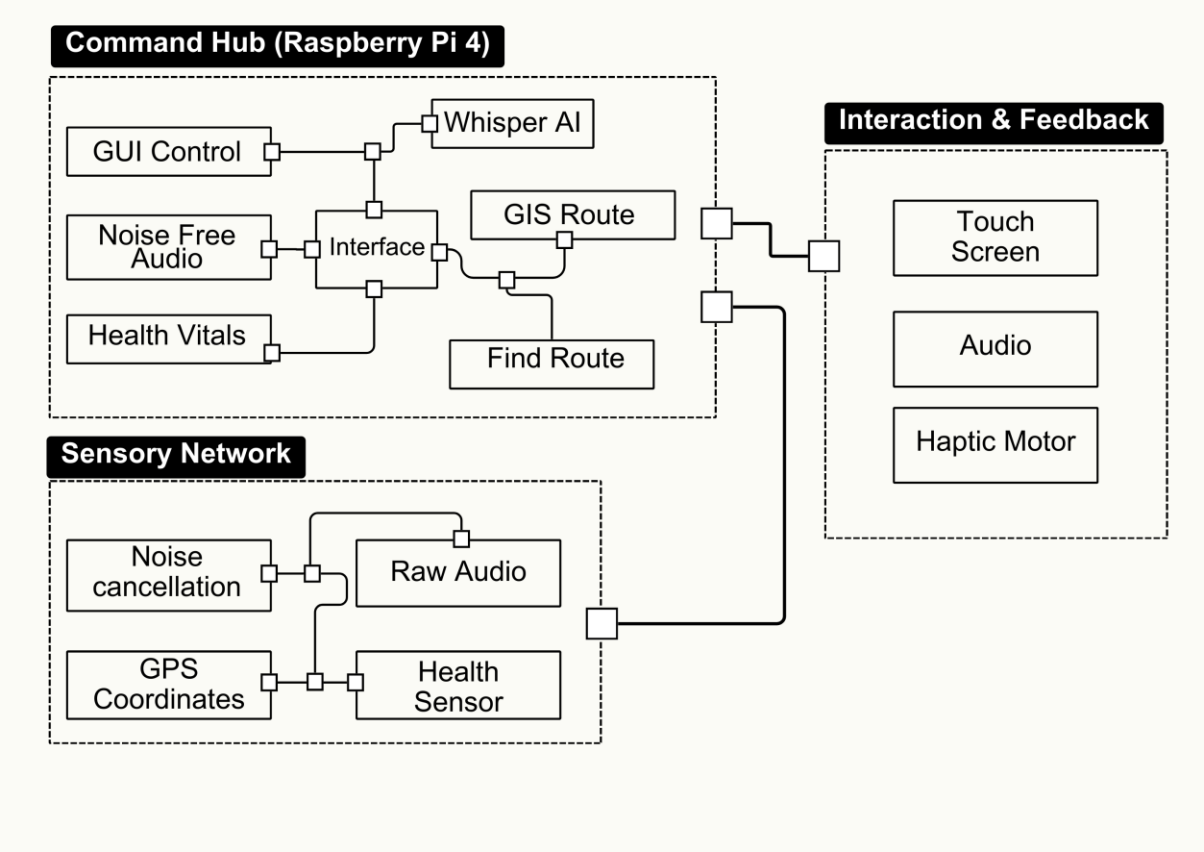


Figure 6: Component Diagram for Auralaid

5.2.3 Data Flow Diagrams

Data Flow Diagrams (DFDs) illustrate how information moves through the Auralaid ecosystem, from raw sensory input at the hardware level to processed bionic feedback delivered via the user interface. These diagrams define the transformation of data—such as converting raw audio into text or accelerometer spikes into SOS alerts—as it passes between the user, the embedded hub, and local storage.

Data Flow Diagram Level 0

The Level 0 DFD illustrates the high-level interaction between the Auralaid Bionic Hub and its external entities. The user provides raw audio (speech) and physiological signals (heart rate/SpO2) to the system. The internal AI engine processes these inputs and returns real-time bionic feedback, such as live captions and health waveforms. External entities, such as GPS Satellites and AI Knowledge Bases (Wikipedia/Qwen), provide positional data and contextual information to enhance the system's "Virtual Doctor" and "Navigation" features.i.e., show in Figure 3.

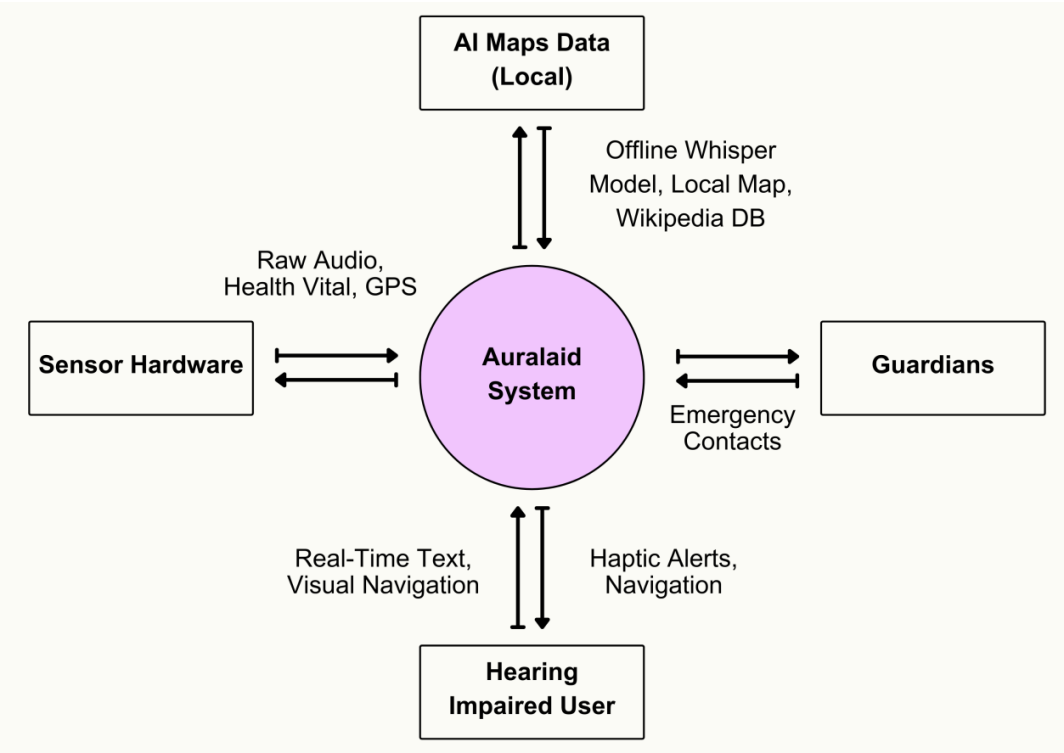


Figure 3: Data Flow Diagram of Auralaid Level 0

Data Flow Diagram Level 1

The Level 1 DFD breaks down the bionic hub into its core functional processes: Transcription, Biometric Analysis, and Safety Protocol Management.

Raw I2S audio is fed into the Whisper AI model; the resulting text is formatted for the GUI and logged to local session files.

I2C data from health sensors is filtered and analyzed; if vitals exceed safe thresholds, a "Health Alert" is triggered.

IMU data is continuously monitored for fall signatures; upon detection, the system fetches the current GPS coordinates and initiates the SOS broadcast. The system ensures all processed data is stored in the Local Telemetry Store (SD Card) for historical review., show in Figure 4.

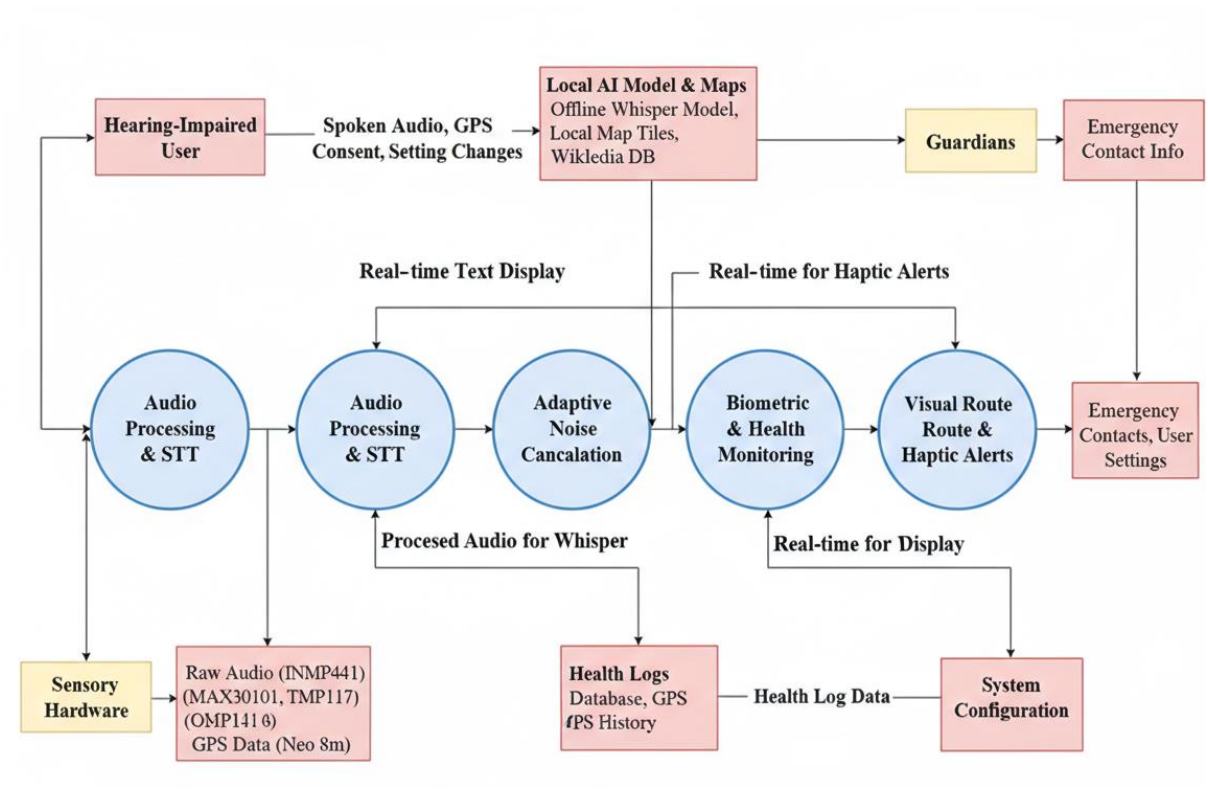


Figure 4: Data Flow Diagram for Auralaid Level 1

Data Flow Diagram Level 2

The Level 2 DFD details the specific data transformations within the bionic subsystems. For example, in the Guardian Mode sub-process X, Y, Z acceleration data is sampled at high frequency. Analysis: The system calculates the resultant G-force and monitors for a impact followed by zero-motion. The system queries the Local User Profile for emergency contacts and the Neo 8m Module for a GPS lock. An encrypted data packet containing the location and user ID is transmitted via the Mobile Gateway. This granular flow ensures that every sensor event is accurately translated into a meaningful bionic response or safety action. i.e., show in Figure 5.

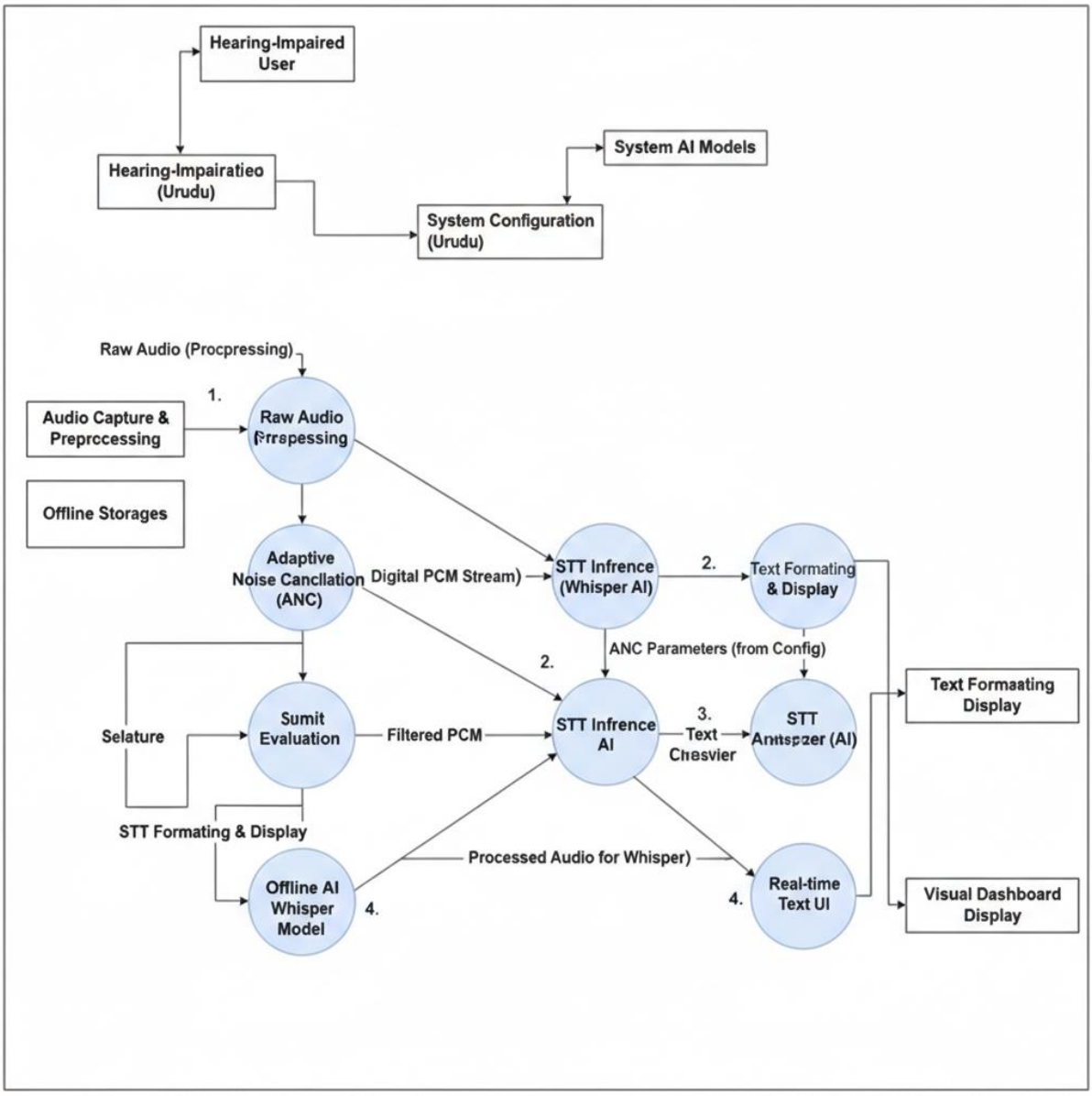


Figure 5: Data Flow Diagram for Auralaid Level 2

5.2.4 Activity Diagram:

This diagram illustrates the overall workflow and architecture of the AuralAid system, showing how data flows from the wearable device to processing units and finally to the user interface. The process starts at the Wearable Node, where raw audio is captured, environmental and health sensors are polled, and GIS coordinates are collected. This data is sent to the Pi Hub, which acts as the central processing unit. In the Pi Hub, acoustic processing is performed, including noise cancellation and local speech-to-text (STT), while vital processing analyzes health-related sensor data. Based on this processing, the system also handles route navigation using location information. The processed outputs are then forwarded to the Interface, where users can access features such as real-time audio streaming,

a health dashboard, safe route guidance, and haptic feedback alerts. If critical conditions are detected, the system triggers notifications through the guardian module. The workflow concludes once all alerts and user interactions are completed, ensuring real-time assistance, safety, and situational awareness for the user.

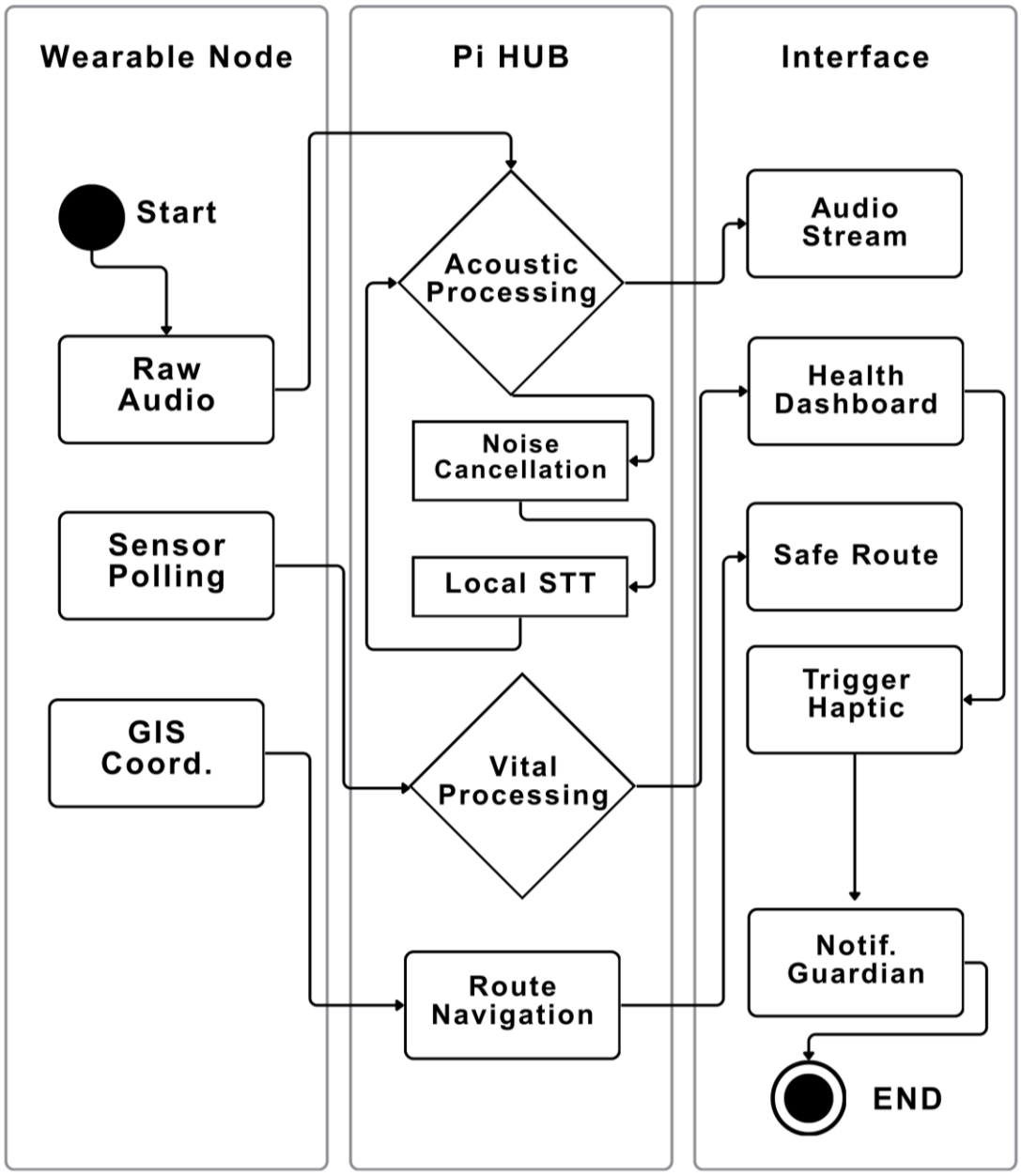


Figure 10: Activity Diagram for Auralaid

5.2.5 Sequence Diagram:

A sequence diagram for the AuralAid project represents the step-by-step interaction between system components over time to deliver real-time hearing assistance and safety features. The sequence begins when the user activates the wearable device, which captures raw audio and sensor data and sends it to the processing unit (Pi Hub). The Pi Hub performs acoustic processing such as noise cancellation and speech-to-text conversion, along with vital data analysis from health sensors. The processed information is then sent to the mobile interface, where it is displayed as audio streams, captions, health status, or navigation guidance. If abnormal noise levels, health risks, or unsafe conditions are detected, the system immediately triggers haptic feedback and notification alerts. This sequence diagram clearly shows the ordered flow of messages and actions among the wearable node, processing unit, and user interface, ensuring timely responses, system coordination, and effective assistance in the AuralAid system.

1. **Sequence Diagram of Real time audio processing:**

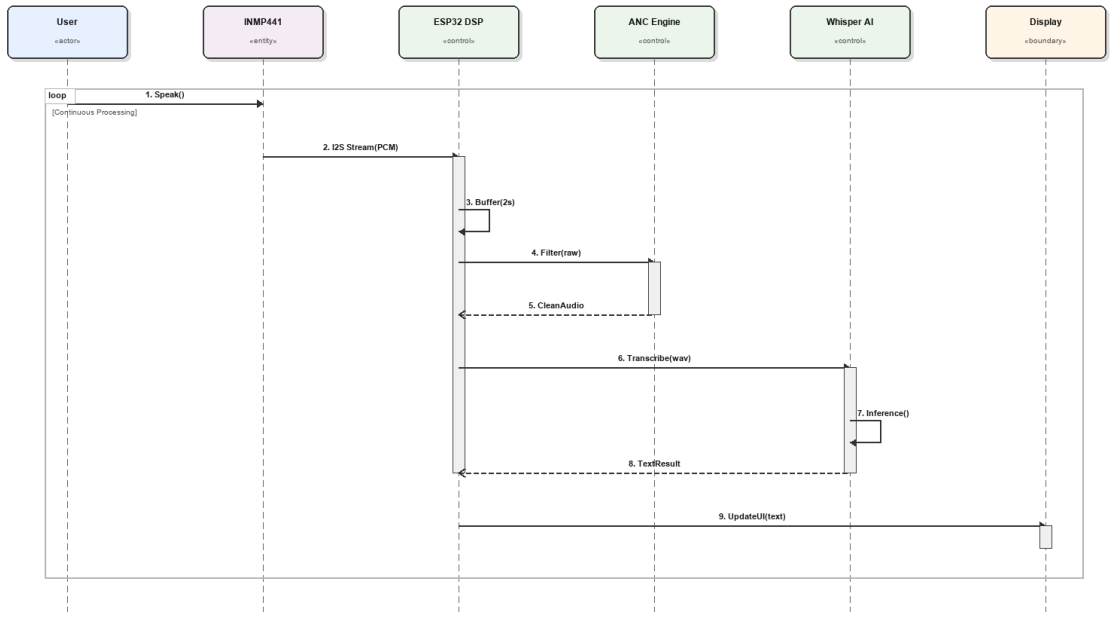


Figure 11: Sequence Diagram of Real time audio processing

2. Sequence Diagram of Biometric Health monitoring:

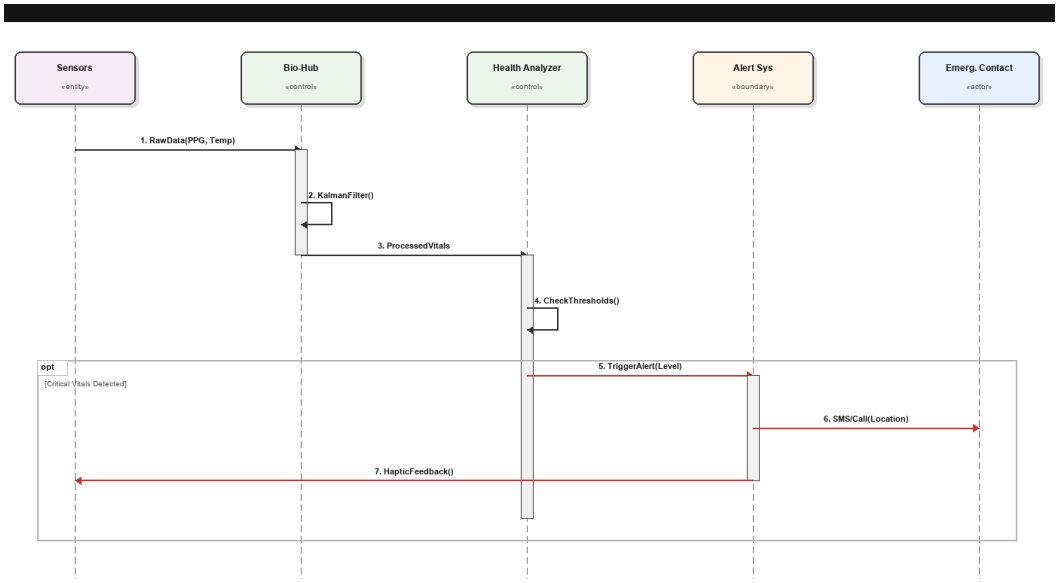


Figure 12: Sequence Diagram of Biometric Health monitoring

3. Sequence Diagram of full system integration:

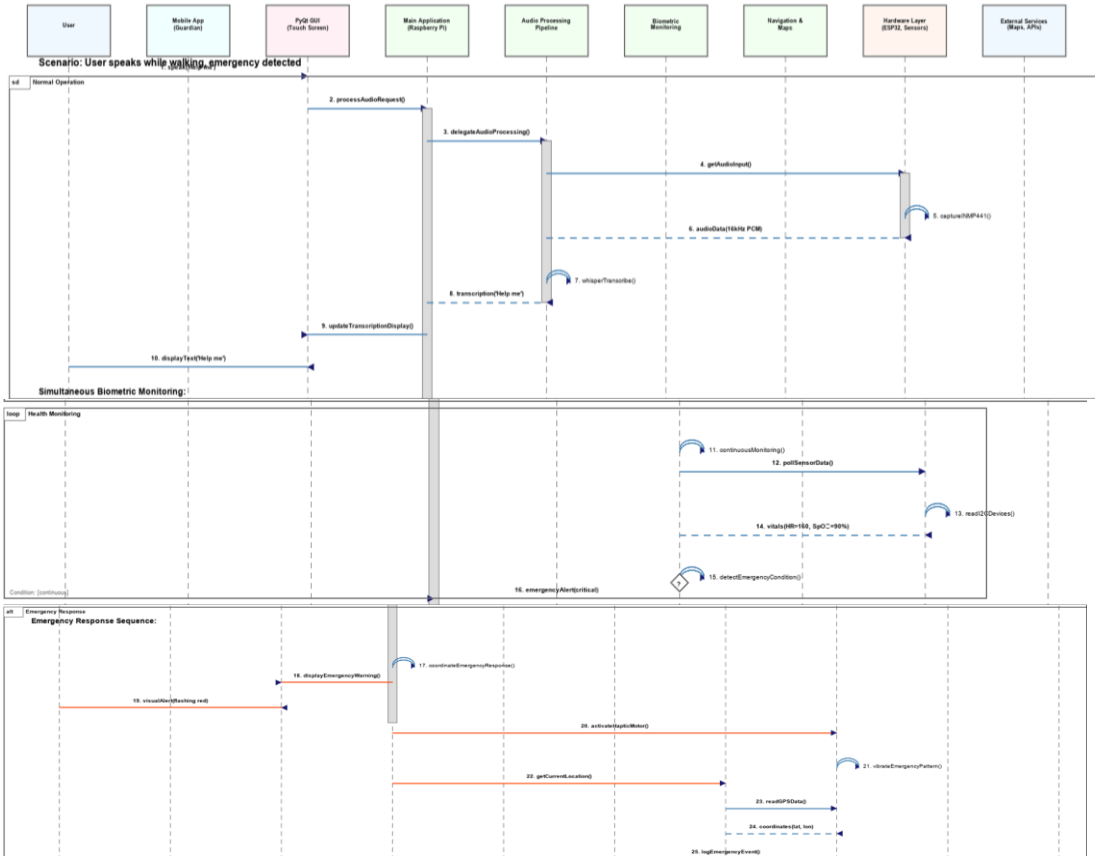
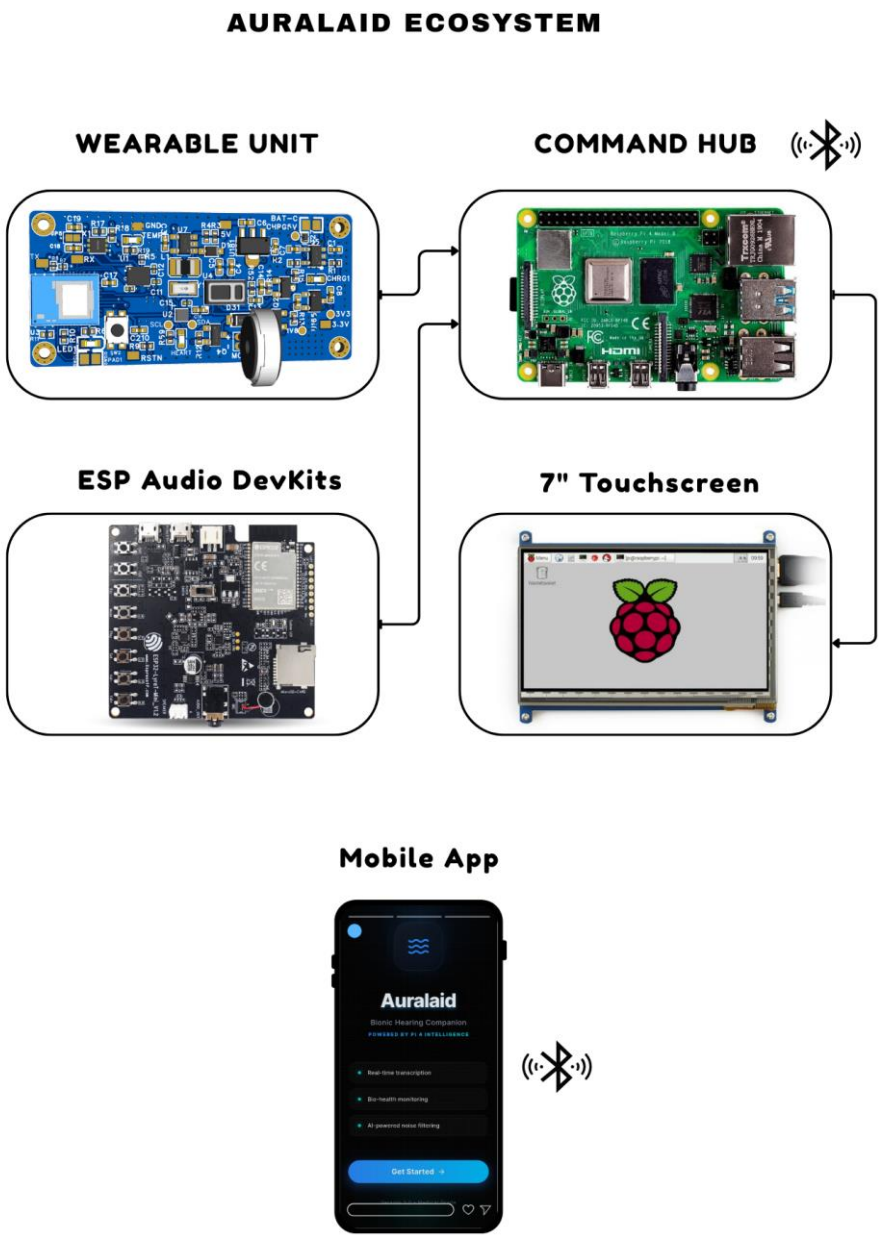


Figure 13: Sequence Diagram of full system integration

5.2.6 AuralAid Ecosystem:

Architecture of the AuralAid Ecosystem, the wearable unit captures audio and sensor data, while the ESP Audio Devkits handle audio processing tasks. The Command Hub, based on a Raspberry Pi, performs central control and data management and provides user interaction through a 7-inch touchscreen. The mobile application connects to the system via Bluetooth, allowing users to monitor system status, receive alerts, and manage hearing assistance features in real time.



5.2.7 State Machine Diagram:

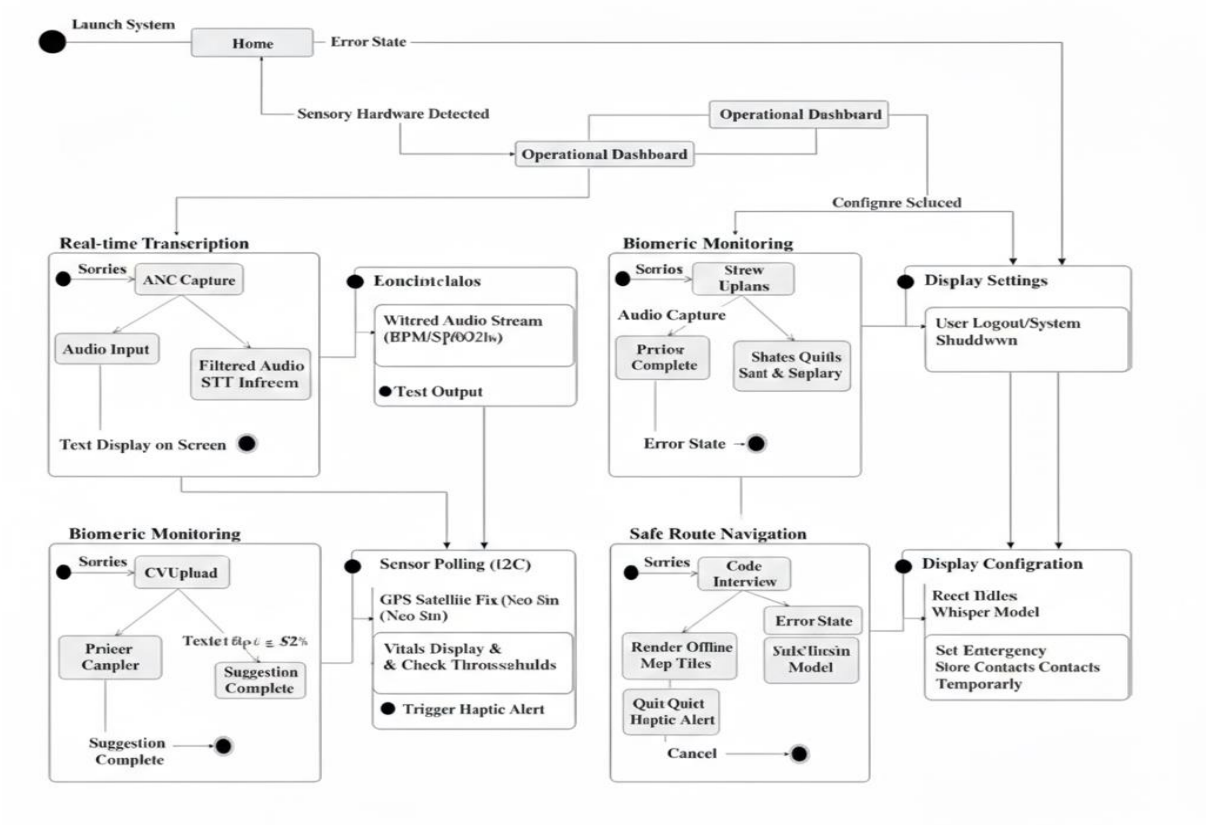


Figure 23: State Machine Diagram for Auralaid System

5.2.8 GUI

The following figures illustrate the user interface of the AuralAid mobile application, designed to provide an intuitive and accessible experience for users with hearing assistance needs. The interface focuses on simplicity, real-time feedback, and quick access to critical features such as device connectivity, audio transcription, noise filtering, health monitoring, and emergency alerts. Each screen is designed with a minimal layout and high-contrast elements to ensure usability, clarity, and fast interaction during real-world usage.

1. AuralAid Application Splash Screen:

Displays the app introduction and highlights core features such as real-time transcription, bio-health monitoring, and AI-powered noise filtering.

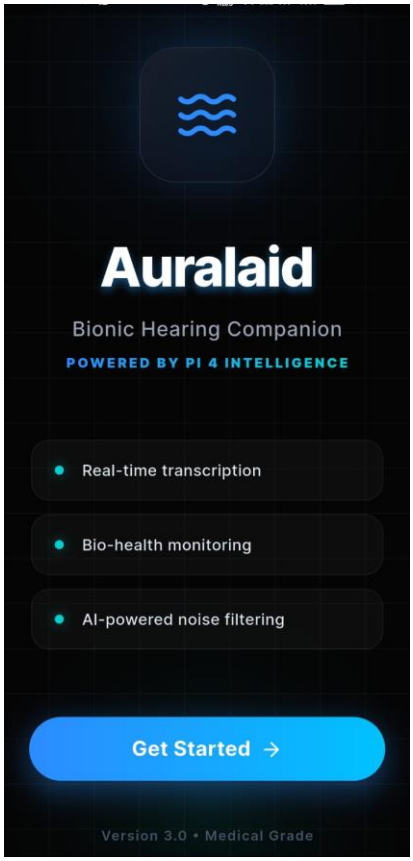


Figure 14: Splash Screen

2. Device Discovery and Bluetooth Connectivity Screen:

Shows available audio and wearable devices and allows users to establish a secure connection.

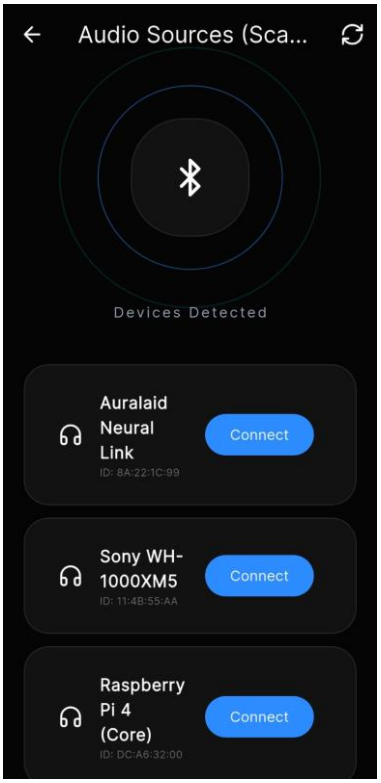


Figure 15: Connectivity Screen

3. User Profile and Emergency Contact Setup Screen:

Allows users to manage personal information and configure emergency protocols.

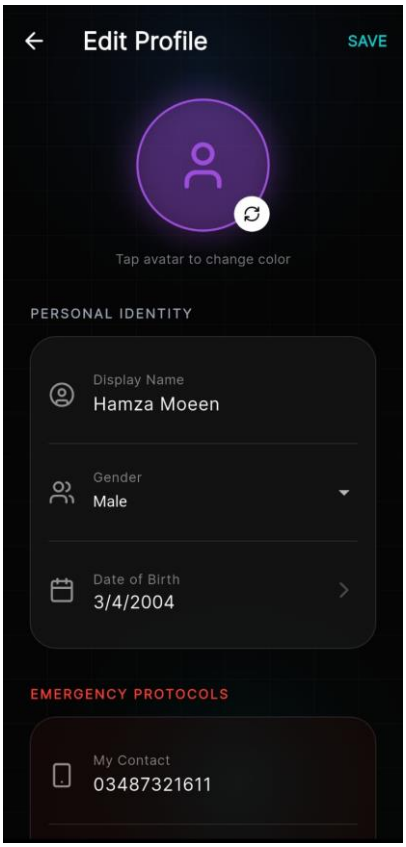


Figure 16: User Profile and Emergency Contact Setup Screen

4. AuralAid Main Dashboard:

Displays system status, noise levels, quick access modules, and emergency SOS option.

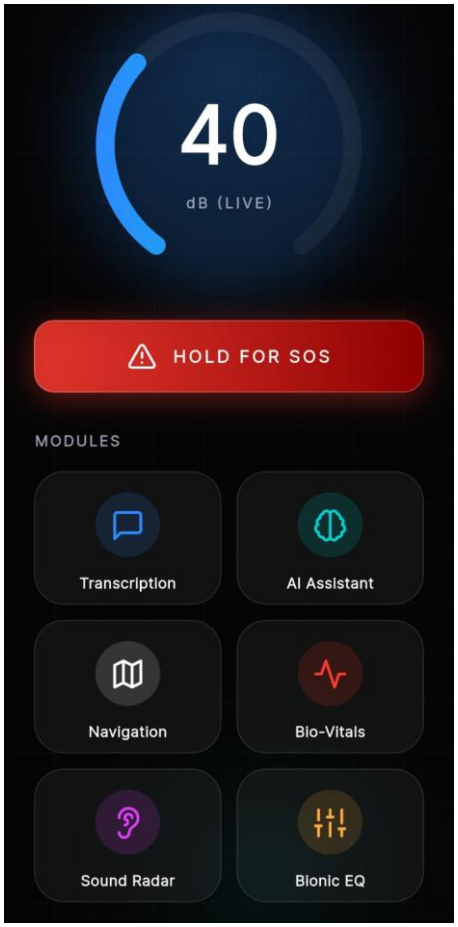


Figure 17: AuralAid Main Dashboard

6. Real-Time Audio Transcription Interface:

Shows live transcription output with system listening status and volume controls.

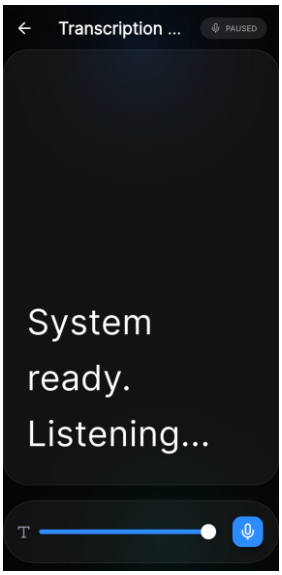


Figure 18: Real-Time Audio Transcription Interface

7. Open mind AI:

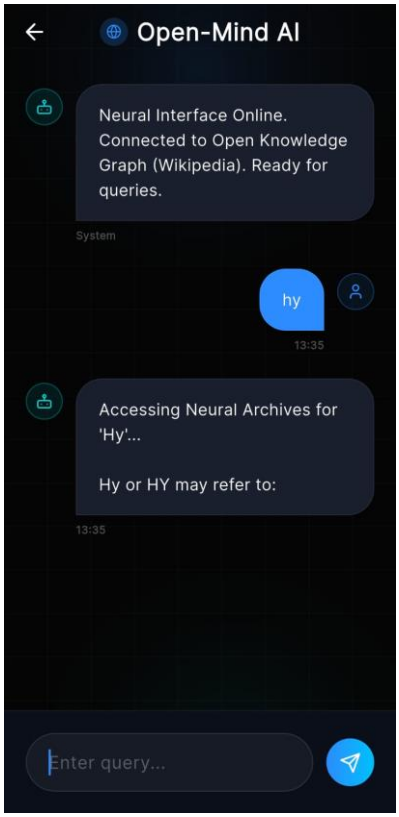


Figure 19: Open mind AI

8. Google map for guidance and alert:

Show Rute that are safe and if there are too much traffic than they send alert.

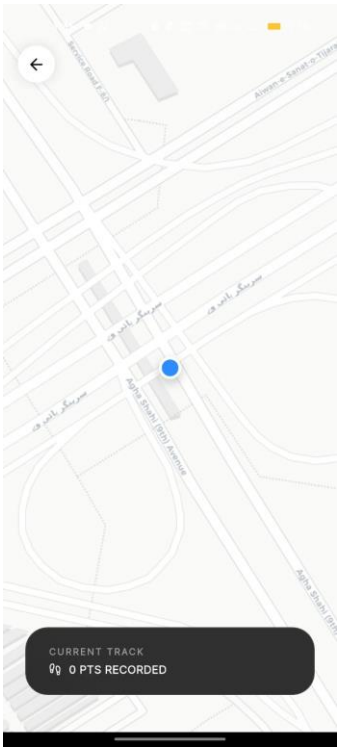


Figure 20: Google map

9. Bio-Metric Screen:

Show the person health condition currently and if the person health condition are abnormal than they send alert on mobile and device.

Figure 21: Bio-Metric Screen

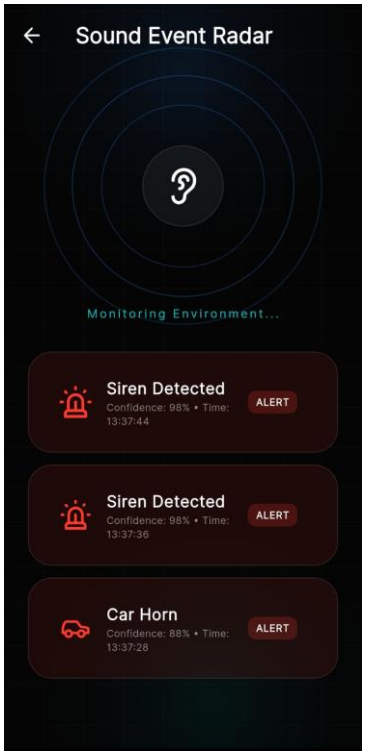


Figure 22: Sound Event Radar

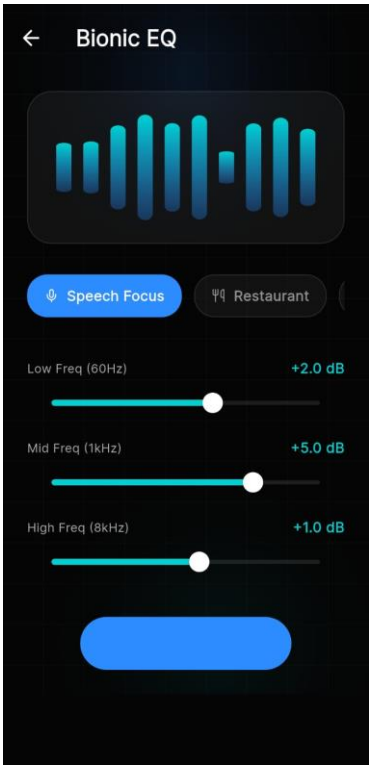


Figure 23: Bionic EQ

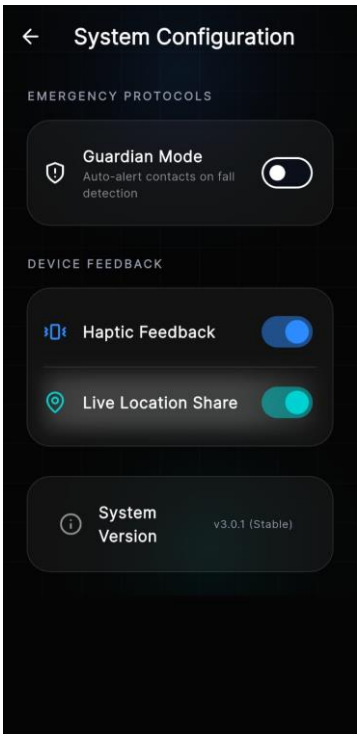


Figure 24: System Configuration settings

9.1.1 Behavioral Design

The Behavioral Design phase defines the dynamic interactions of the Auralaid system. It ensures that the platform responds correctly to various input scenarios such as a sudden spike in ambient noise or a user's heart rate falling outside of safe limits.

Key Behavioral Scenarios:

The system transcribes audio and polls vitals at 1Hz, updating the dashboard with smoothed waveforms.

If the MPU6050 detects a 3.5G impact followed by immobility, the system interrupts all standard background tasks to initiate the Guardian Mode SOS sequence.

If a sensor (e.g., the MAX30101) disconnects, the GUI behaviorally shifts to an "Alert State," providing a visual haptic notification and a troubleshooting guide.

9.1.2 Use Case Diagram

The Auralaid Use Case Diagram illustrates the high-level functional relationships between the User (Primary Actor) and the Bionic Hub. It provides an operational standpoint for major tasks, such as managing health logs, viewing real-time transcriptions, and configuring safety alerts.

The diagram maps out three critical functional zones:

- Real-time STT (English/Urdu) and Vocal Sentiment Analysis.
- Fall detection, GPS breadcrumb tracking, and Emergency SMS.
- Local profile encryption, session archiving, and system-wide "Privacy Shield" toggling.

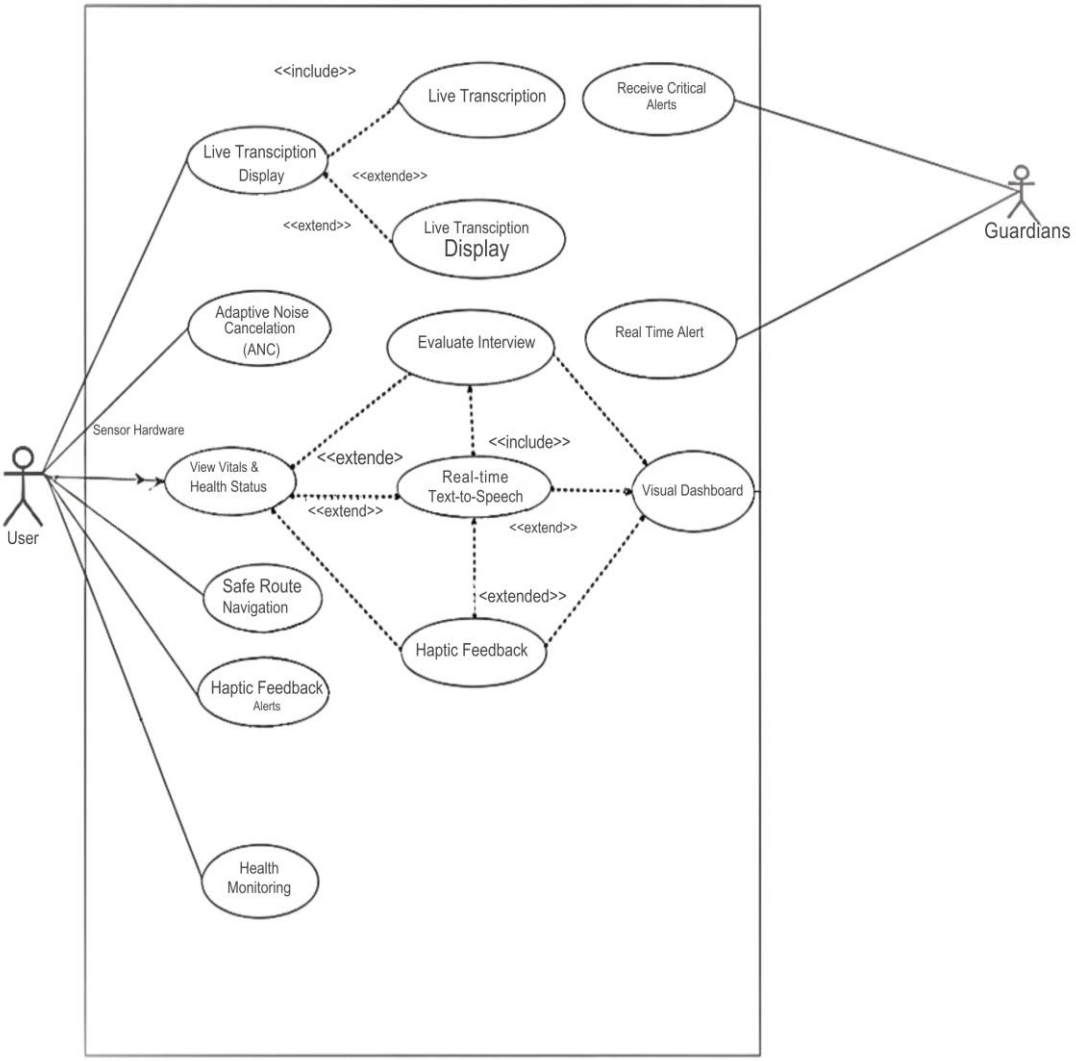


Figure 25: Use case Diagram for Aural Aid

10. Software Planning and Timeline

Software Planning and Timeline are critical components of the AuralAid project, aimed at defining how the adaptive AI-based hearing assistance system will be developed, tested, and delivered. The planning phase helps in identifying the overall scope, milestones, and specific objectives that must be achieved at different stages of the project. A well-structured timeline ensures that development activities are carried out in a resource-efficient and time-bound manner, allowing the team to systematically achieve the project goals. The AuralAid project timeline is divided into distinct phases, each associated with specific tasks and deliverables to ensure smooth progress and successful completion of the system.

10.1 Work Breakdown Structure

The Work Breakdown Structure (WBS) provides a systematic decomposition of the AuralAid project into manageable and well-defined tasks. It serves as a roadmap that guides the development process by breaking down complex activities into smaller components with clear deliverables. The WBS supports effective time management, optimal utilization of resources, and risk mitigation throughout the development lifecycle of AuralAid.

Phases of Development:

The AuralAid system is developed through the following key phases:

Requirement Analysis:

This phase focuses on identifying and documenting the core functional and non-functional requirements of AuralAid, including real-time audio amplification, noise detection and alerts, speech-to-text conversion, emotion recognition, health monitoring features, and device mobile application interaction.

System Design:

In this phase, the overall system architecture is designed, including the wearable device structure, mobile application flow, AI processing modules, and user interface design. Hardware components such as microphones and sensors are also planned, along with software module interactions.

Development:

This phase involves the implementation of the AuralAid system. It includes development of the mobile application interface, integration of AI models for speech processing and emotion

detection, implementation of noise alert mechanisms, and firmware development for the wearable device.

Testing:

Testing ensures the reliability and accuracy of the AuralAid system. Functional testing, usability testing, performance testing, and device interaction testing are conducted to verify that all features work as intended and meet user requirements.

Deployment and Maintenance:

In the final phase, the system is prepared for demonstration and deployment. Bug fixes, performance optimization, and final refinements are performed. Maintenance activities ensure system stability and readiness for future enhancements.

2. Task Breakdown and Deliverables

Each development phase is further divided into smaller tasks with clearly defined deliverables:

- **Mobile Application Development:** Design and implementation of user interfaces such as dashboard, audio captions, noise alerts, and settings screens.
- **Device & Firmware Development:** Integration of sensors, microphones, and communication modules with stable firmware.
- **AI Module Integration:** Implementation of speech-to-text processing, noise filtering, and emotion detection models.
- **Testing Deliverables:** Test cases, test reports, and validation results for system accuracy and usability.
- **Final Documentation:** Complete project report, user manual, and system demonstration materials.

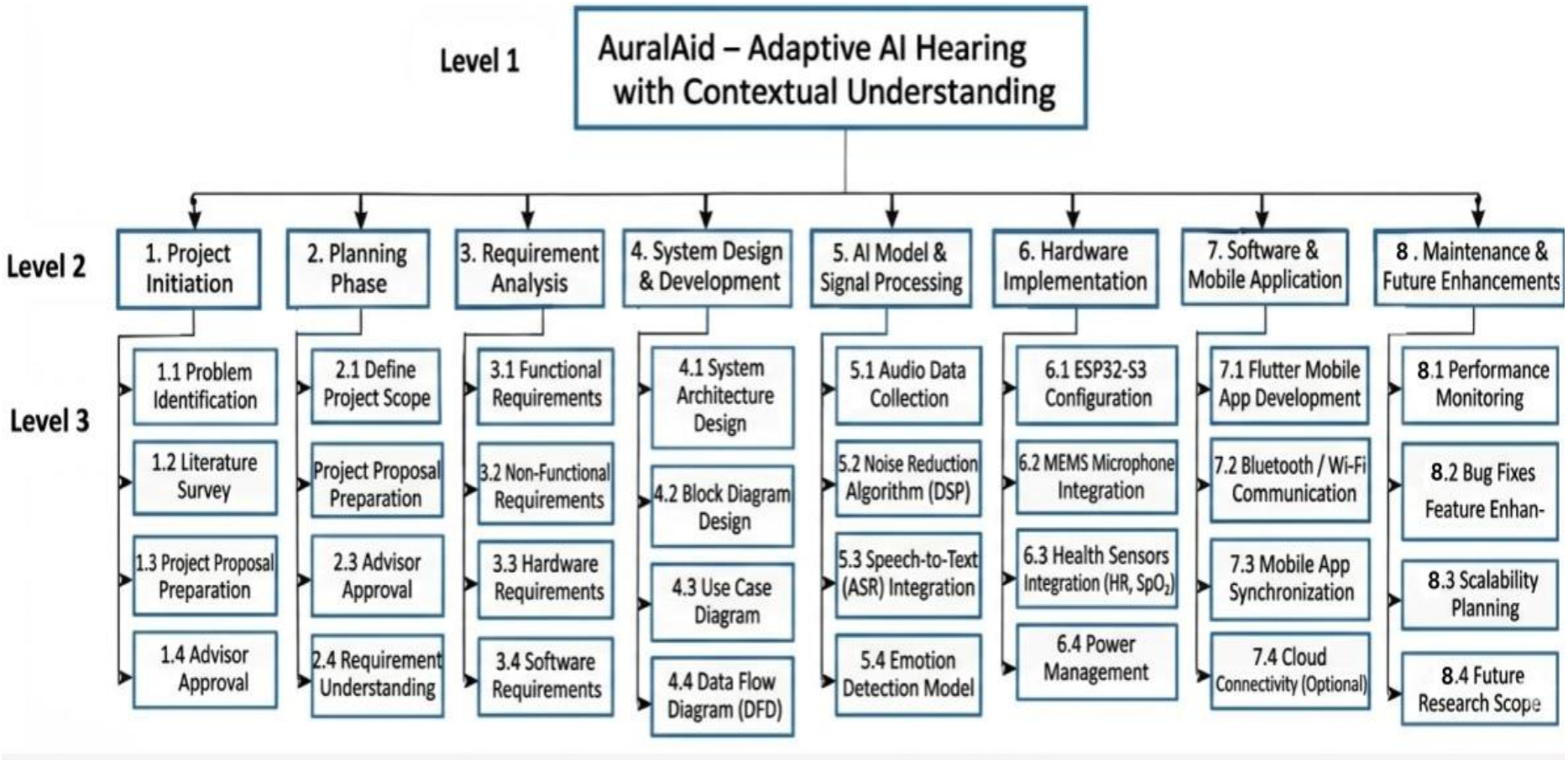


Figure 28 : Work break Down Structure of AuralAid

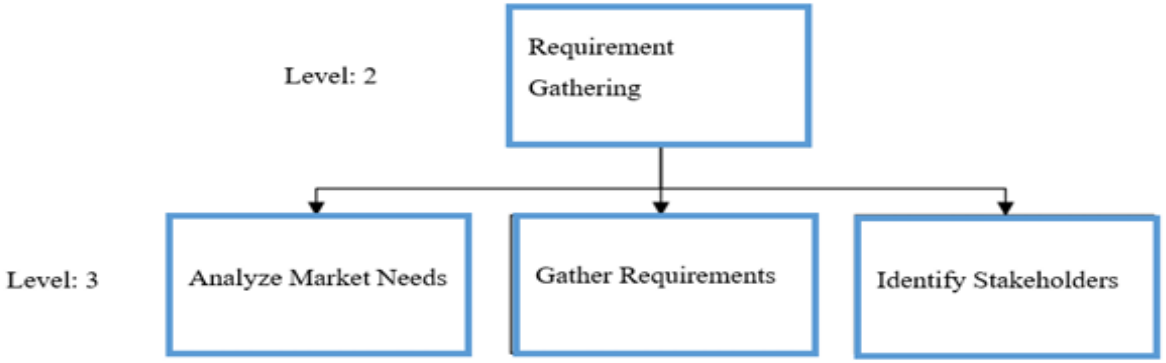


Figure 29: Requirement Gathering

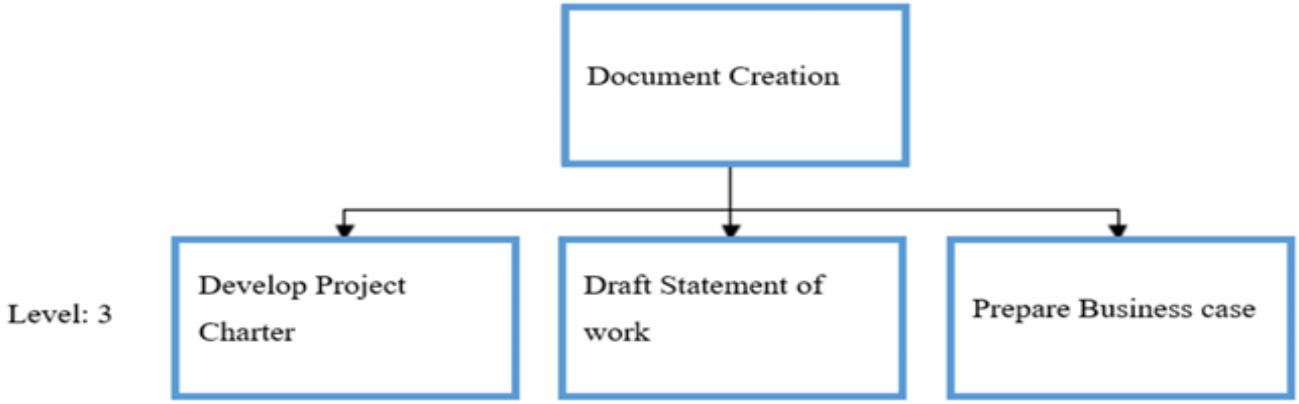


Figure 30: Document Creation

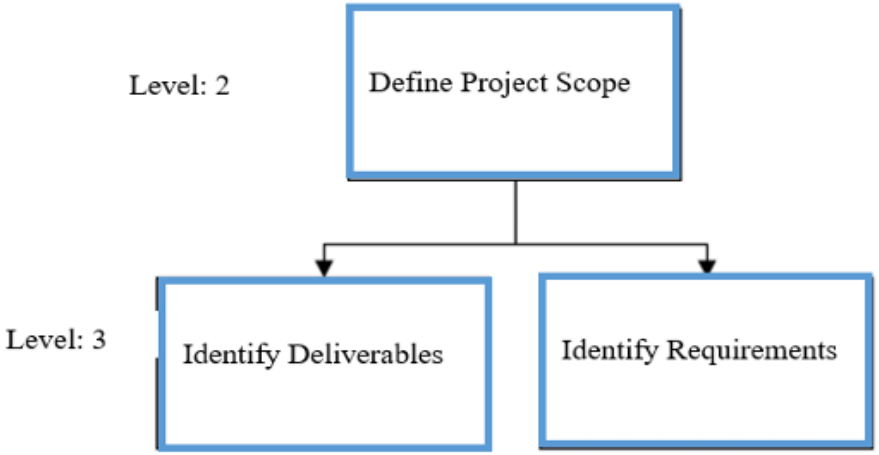


Figure 31: Define Project Scope

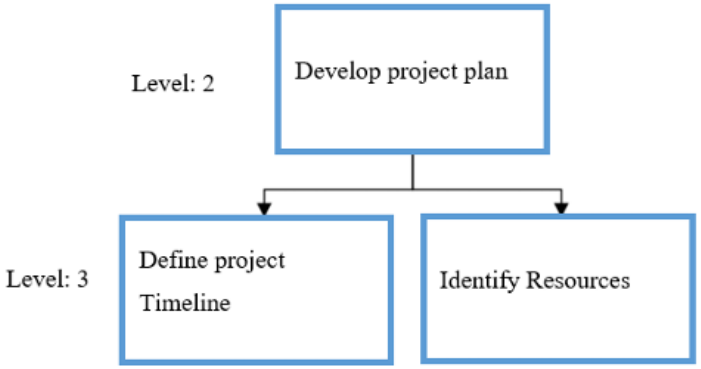


Figure 32: Develop Project Plan

Figure 33: Risk Management Planning

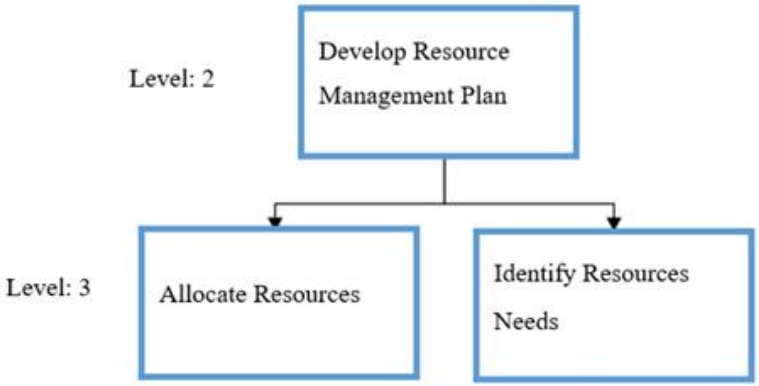


Figure 34: Develop Resource Management Plan

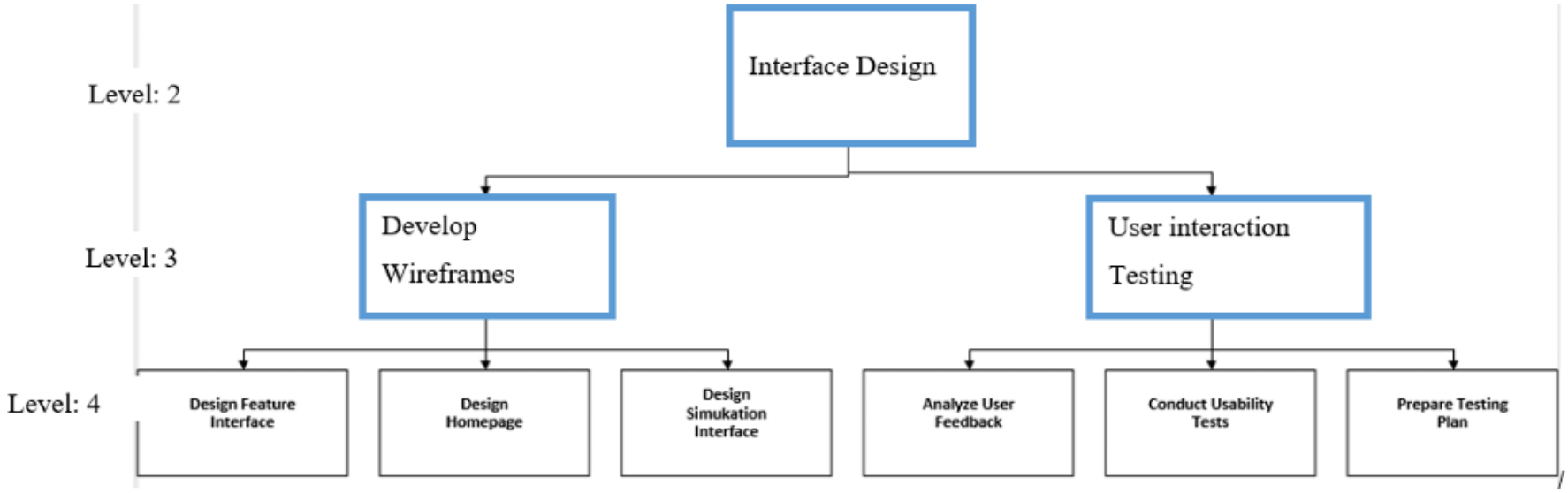


Figure 35: Interface Design

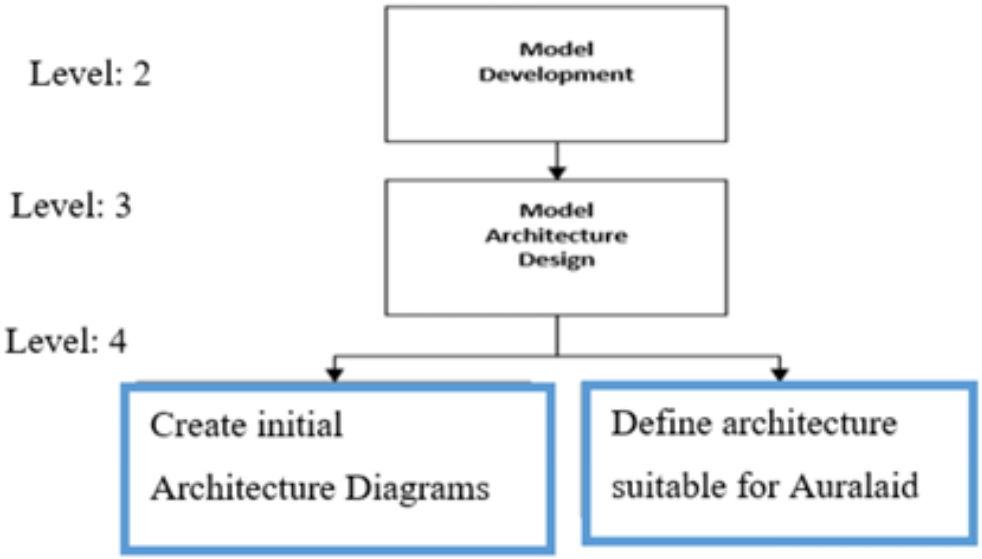


Figure 36: Model Development

7.2 Milestones and Deliverables

The sections titled Milestones and Deliverables summaries the major stages of the project and the major deliverables and documents that we generate at each stage. They serve the purposes of the markers of implementing various milestones to monitor our progress and make a surety that all the phases will bring the results that the project requires.

Table 27: Milestones and Deliverables of AuralAid

Task Name	Start Date	End Date
1. Project Initiation	Mon 10/1/25	Mon 10/28/25
Business Case	Mon 10/28/25	Mon 10/28/25
Statement of Work	Mon 10/28/25	Mon 10/28/25
Project Charter	Mon 10/28/25	Mon 10/28/25
Project Proposal	Mon 10/28/25	Mon 10/28/25
2. Planning Phase	Thu 10/29/25	Thu 11/18/25
Requirement Document	Thu 11/1/25	Thu 11/1/25
Work Breakdown Structure	Sun 11/8/25	Sun 11/8/25
Risk Management Plan	Wed 11/13/25	Wed 11/13/25
Resource Allocation Document	Mon 11/18/25	Mon 11/18/25
3. Design and Development	Mon 11/19/25	Mon 1/24/26
Figma Prototype	Mon 1/24/26	Mon 1/24/26
System Architecture Document	Mon 1/24/26	Mon 1/24/26
Detailed Use case Document	Mon 1/24/26	Mon 1/24/26
4. AI Model Training	Fri 1/21/26	Fri 3/23/26
Data Collection Report	Fri 3/23/26	Fri 3/23/26
5. Testing and Quality Assurance	Mon 3/24/26	Mon 5/17/26
Test Cases Document	Mon 5/17/26	Mon 5/17/26
Test Problem Report	Mon 5/17/26	Mon 5/17/26
Integration Test Plan	Mon 5/17/26	Mon 5/17/26
Integration Test Results	Mon 5/17/26	Mon 5/17/26
UAT Plan	Mon 5/17/26	Mon 5/17/26
UAT Feedback Report	Mon 5/17/26	Mon 5/17/26
6. Deployment	Thu 5/16/26	Thu 6/2/26
Deployment And Launch Report	Thu 6/2/26	Thu 6/2/26
7. Maintenance and Updates	Fri 5/30/26	Fri 6/6/26
Maintenance and Enhancement Report	Fri 6/6/26	Fri 6/6/26

10.2 Detailed Baseline Plan

In the Baseline Plan of AuralAid the project, the process to tackle is divided into SDLC phases which define the work from identification to implementation. Every of them corresponds to certain goals.

	Task Mode	Task Name	Duration	Start	Finish
1		**Adaptive AI Hearing with Contextual Understanding**	**9 mons**	**Wed 10/1/25**	**Tue 6/9/26**
2		Project Initiation	1 mon	Wed 10/1/25	Tue 10/28/25
3		Planning Phase	0.75 mons	Wed 10/29/25	Tue 11/18/25
4		Design Aand Development	2.45 mons	Tue 11/18/25	Sun 1/25/26
5		AI Model Traning	45 days	Mon 1/26/26	Fri 3/27/26
6		Testing and Quality Assurance	41 days	Mon 3/30/26	Mon 5/25/26
7		Deployment	12 days	Tue 5/26/26	Wed 6/10/26
8		Maintenance and Updates	7 days	Mon 6/1/26	Tue 6/9/26

Figure 40: Gantt Chart for AuralAid

1. Project Initiation (10/1/2025 - 10/28/2025):

The Project Initiation phase, which occurs from is from October 1, 2025 to October 28, 2025, represent the first phase of the project. This phase comprises identification of the stakeholders, requirements, assessment of the market, preparation of the business case and development of the Statement of Work. The manager has to clearly elucidate the goals of the project as well as its parameters and goals.

	Task Mode	Task Name	Duration	Start	Finish	Pred
1		**Adaptive AI Hearing with Contextual Understanding**	**9 mons?**	**Wed 10/1/25**	**Tue 6/9/26**	
2		**1 Project Initiation**	**1 mon?**	**Wed 10/1/25**	**Tue 10/28/25**	
3		**1.1 Requirement Gathering**	**1 mon?**	**Wed 10/1/25**	**Tue 10/28/25**	
4		1.1.1 Identify Stakeholders	0.2 mons	Wed 10/1/25	Mon 10/6/25	
5		1.1.2 GatherRequirement	0.2 mons	Tue 10/7/25	Fri 10/10/25	4
6		1.1.3 Analyze Market Need	0.1 mons	Mon 10/13/25	Tue 10/14/25	5
7		**1.2 Document Creation**	**0.55 mons**	**Tue 10/14/25**	**Tue 10/28/25**	
8		1.2.1 Prepre Business Case	0.25 mons	Wed 10/15/25	Tue 10/21/25	
9		1.2.2 Draft Statement	0.15 mons	Tue 10/21/25	Thu 10/23/25	
10		1.2.3 Develop Project Chart	0.15 mons	Fri 10/24/25	Tue 10/28/25	9
11		**1.3 <Business Case>**	0 mons	Tue 10/28/25	Tue 10/28/25	
12		**1.4 <Statement of Work>**	0 mons	Tue 10/28/25	Tue 10/28/25	11
13		**1.5 <Project Charter>**	0 mons	Tue 10/28/25	Tue 10/28/25	12
14		**1.6 <Project Proposal>**	0 days	Tue 10/28/25	Tue 10/28/25	13

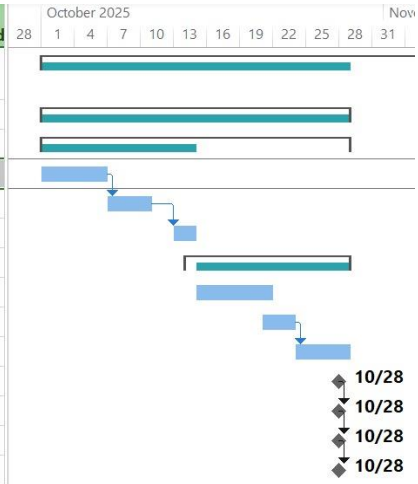


Figure 41: Gantt Chart for Project Initiation

2. Planning Phase (October 29, 2025 - November 18, 2025):

Planning takes place from October 29, 2025 to November 18, 2025 and involves the identification of project work, objectives, resources which include personnel, equipment and time. They include setting up project schedule, risk analysis, resource management and risk

management strategy. This phase makes sure that the project is correctly positioned for execution for its objectives to be met as planned.

	Task Mode	Task Name	Duration	Start	Finish
1		**Adaptive AI Hearing with Contextual Understanding**	**9 mons?**	**Wed 10/1/25**	**Tue 6/9/26**
2		**1 Project Initiation**	1 mon	Wed 10/1/25	Tue 10/28/25
3		**2 Planing Phase**	**16 days**	**Wed 10/29/25**	**Wed 11/19/25**
4		**2.1 Define Project Scope**	**4 days**	**Wed 10/29/25**	**Tue 11/4/25**
5		2.1.1 Identify Deliverables	0.15 mons	Wed 10/29/25	Fri 10/31/25
6		2.1.2 Identify Requirements	0.05 mons	Mon 11/3/25	Mon 11/3/25
7		2.1.3 < Requirement Docunentation>	0 mons	Tue 11/4/25	Tue 11/4/25
8		**2.2 Develop Project Plan**	**6 days**	**Sun 11/2/25**	**Tue 11/11/25**
9		2.2.1 Define Project Timeline	0.15 mons	Sun 11/2/25	Tue 11/4/25
10		2.2.2 Identify Resources	0.2 mons	Wed 11/5/25	Mon 11/10/25
11		2.2.3 <Work Breakdown Structure>	0 mons	Tue 11/11/25	Tue 11/11/25
12		**2.3 Risk Management Planing**	**5 days**	**Wed 11/12/25**	**Wed 11/19/25**
13		2.3.1 Identify Risks	0.1 mons	Wed 11/12/25	Thu 11/13/25
14		2.3.2 Analyze Risks	0.1 mons	Fri 11/14/25	Mon 11/17/25
15		2.3.3 Develop Mitigation	0.05 mons	Tue 11/18/25	Tue 11/18/25
16		2.3.4 <Risk Management planning>	0 mons	Wed 11/19/25	Wed 11/19/25
17		**2.4 Develop Resource Management**	**4 days**	**Fri 11/14/25**	**Wed 11/19/25**
18		2.4.1 Identify Resource	0.15 mons	Fri 11/14/25	Tue 11/18/25

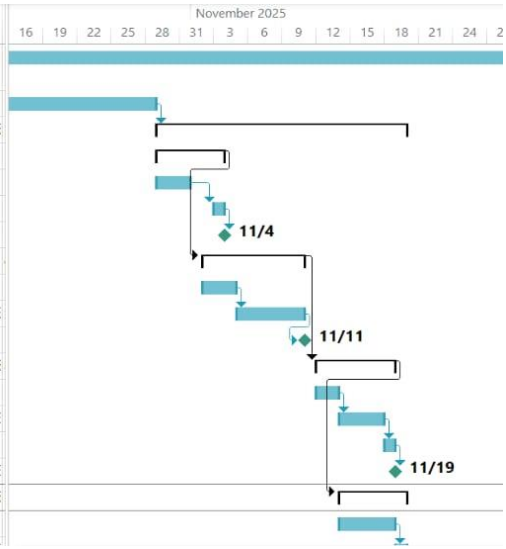


Figure 42: Gantt Chart for Planning

3. AI Model Training (1/21/2026 - 3/23/2026)

The AI Model Training phase from January 21, 2026, to March 23, 2026, include the data collection and preprocessing and model training. This phase involves indexing of datasets, preprocessing of data collected.

	Task Mode	Task Name	Duration	Start	Finish
1		**Adaptive AI Hearing with Contextual Understanding**	**9 mons?**	**Wed 10/1/25**	**Tue 6/9/26**
2		**1 Project Initiation**	1 mon	Wed 10/1/25	Tue 10/28/25
3		**2 Planning Phase**	0.75 mons	Wed 10/29/25	Tue 11/18/25
4		**3 Design And Development**	2.45 mons	Wed 11/19/25	Mon 1/26/26
5		**4 AI Model Traning**	**45 days?**	**Wed 1/21/26**	**Tue 3/24/26**
6		**4.1 Data Collection**	**24 days**	**Wed 1/21/26**	**Mon 2/23/26**
7		4.1.1 Candidatre Profile	0.5 mons	Wed 1/21/26	Tue 2/3/26
8		4.1.2 Industry Analysis Data	0.7 mons	Tue 2/3/26	Fri 2/20/26
9		4.1.3 <Data Collection>	0 days	Sat 2/21/26	Sat 2/21/26
10		**4.2 Model Development**	**0.35 mons**	**Sat 2/21/26**	**Sat 2/28/26**
11		4.2.1 Model Architecture Design	0.35 mons	Sat 2/21/26	Sat 2/28/26
12		**4.3 Algorithm Traning**	**7 days**	**Fri 2/27/26**	**Sun 3/8/26**
13		4.3.1 Develop Traning Data	0.15 mons	Fri 2/27/26	Tue 3/3/26
14		**4.4 Evaluation And Tuning**	**0.6 mons?**	**Mon 3/9/26**	**Tue 3/24/26**
15		4.4.1 Create Performance	0.2 mons	Thu 3/19/26	Tue 3/24/26

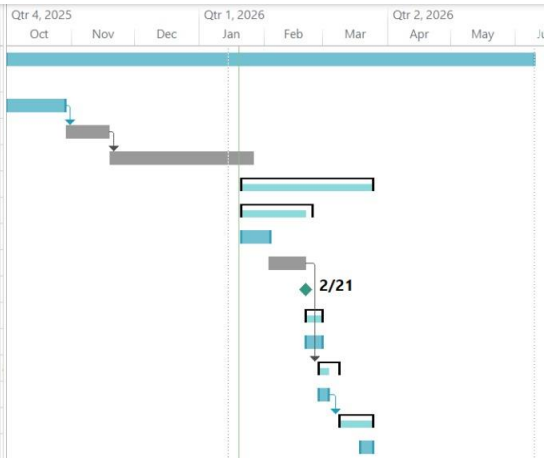


Figure 43: Gantt Chart for AI Model Training

4. Testing and Quality Assurance (5/30/2026 - 6/9/2026)

Testing and Quality Assurance phase is the critical stage of the project that is scheduled from March 24, 2026, to May 17, 2026.

	Task Mode	Task Name	Duration	Start	Finish
17		**7 Maintanance and Updates**	**8 days?**	**Sat 5/30/26**	**Tue 6/9/26**
18		**7.1 User Support**	**6 days?**	**Sat 5/30/26**	**Fri 6/5/26**
19		7.1.1 Suport Resources	**6 days?**	**Fri 5/29/26**	**Fri 6/5/26**
20		7.1.2 continuous user Feed	**6 days?**	**Fri 5/29/26**	**Fri 6/5/26**
21		**7.2 Platform Enhancements**	**6 days?**	**Sat 5/30/26**	**Fri 6/5/26**
22		7.2.1 Feature Updates	**6 days?**	**Fri 5/29/26**	**Fri 6/5/26**
23		7.2.2 AI Model Optimization	**6 days?**	**Fri 5/29/26**	**Fri 6/5/26**
24		**7.3 Bug Fixes And Issue Resolation**	**6 days?**	**Sat 5/30/26**	**Fri 6/5/26**
25		7.3.1 Regular Issue Tracking	**6 days?**	**Fri 5/29/26**	**Fri 6/5/26**
26		7.3.2 Maintainance	**6 days?**	**Fri 5/29/26**	**Fri 6/5/26**

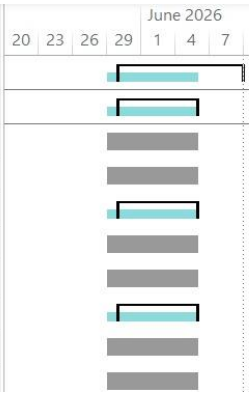


Figure 44 : Gantt Chart for Testing and Quality Assurance

5. Quality Assurance Plan

The Quality Assurance Plan ensures that Aural Aid meets all functional, non-functional, and business requirements, guaranteeing reliability, usability, performance, security, and safety for users with hearing impairment. This plan defines testing requirements, acceptance criteria, and detailed test cases for each major feature of the system.

1.1 Testing Requirements

The testing requirements are designed to comprehensively validate all aspects of Aural Aid, including audio processing, speech-to-text captions, emotion detection, noise alerts, and health monitoring.

8.1.1 Functional Testing

Verify that all features work correctly and satisfy user needs:

1. Capture and process real-time audio efficiently.
2. Convert live speech into text captions with high accuracy.
3. Detect emotions in speech accurately and display them to the user.
4. Generate alerts when environmental noise exceeds safe levels.
5. Track health metrics (heart rate, SpO_2, body temperature) and send notifications.
6. Ensure device pairing, and mobile app connectivity.
7. Ensure proper handling of errors (e.g., sensor disconnections, app crashes).

8.1.2 Performance Testing

Evaluate system performance under various workloads:

1. Ensure low-latency audio processing (<100 ms delay).
2. Ensure speech-to-text conversion responds in near real-time (<2 seconds per phrase).

3. Test simultaneous sensor readings and mobile app updates without lag.
4. Stress test Bluetooth/Wi-Fi connectivity for multiple devices.
5. Measure CPU/memory usage on ESP32-S3 and mobile app during peak operations.

8.1.3 Security Testing

Protect sensitive data and prevent unauthorized access:

1. Test secure storage of user credentials and health data.
2. Ensure encrypted communication between device and mobile app.
3. Test device pairing to prevent unauthorized connections.
4. Check for vulnerabilities in API endpoints.

8.1.4 Integration Testing

Ensure seamless operation of combined modules:

1. Audio capture + noise filtering + speech-to-text modules.
2. Emotion detection + captioning + mobile display.
3. Health sensors + mobile notifications.
4. Mobile app + device synchronization + alert generation.

8.1.5 Usability Testing

Ensure the system is intuitive and user-friendly:

1. Verify app navigation, display of captions, emotions, and alerts.
2. Measure user satisfaction with real-time feedback and health monitoring.
3. Confirm accessibility features: adjustable text size, color contrast, vibration alerts.

8.1.6 Accessibility Testing

Ensure the system can be used by people with disabilities:

1. Confirm captions are readable in all lighting conditions.
2. Verify vibration and LED alerts for users with partial vision.
3. Ensure app supports screen readers and adjustable color schemes.

8.1.7 Compatibility Testing

Verify correct operation across devices:

1. Test mobile app on Android and iOS devices.
2. Test Bluetooth/Wi-Fi connectivity across multiple phone brands.
3. Test audio processing and mobile display under different OS versions.

8.1.8 Beta Testing

Evaluate real-world usability:

1. Conduct trials with hearing-impaired users.
2. Gather feedback on audio clarity, captions, emotion detection, alerts, and mobile app interaction.
3. Collect suggestions for improvement and enhancement.

8.2 Acceptance Criteria

Reference specifications that define what is acceptable in relation to the preparation of Aural Aid for deployment. These criteria make certain that all organizational specifications are met and that the system is stable, well protected, and easy to use.

1.2.1 Functional Acceptance Criteria

1. Audio capture, noise filtering, captions, and emotion detection must work correctly.
2. Health monitoring and alerts must trigger accurately when thresholds are exceeded.
3. Device pairing and app connectivity must function without errors.

8.2.2 Performance Acceptance Criteria

1. Audio processing delay <100 ms.
2. Speech-to-text conversion <2 sec per phrase.
3. Mobile app updates within 1 sec of sensor changes.
4. System handles at least 10 concurrent connected devices without lag.

8.2.3 Security Acceptance Criteria

1. No critical vulnerabilities in app-device communication.
2. Data stored locally and in cloud must be encrypted.
3. Unauthorized pairing attempts must be blocked.

8.2.4 Usability and Accessibility Acceptance Criteria

1. ≥85% user satisfaction in usability testing.
2. Full compliance with accessibility standards (captions, vibration, LED alerts, color contrast).

8.2.5 Integration and Compatibility Criteria

1. All modules and sensors work together without failures.
2. Mobile app functions properly on all tested OS versions and devices.

3. Bluetooth/Wi-Fi connectivity is stable across devices.

8.2.6 Reliability and Recovery Criteria

1. ≥99.9% uptime.
2. Recovery from sensor or connection failure within 5 seconds.

8.2.7 Deployment Readiness Criteria

1. All planned test suites executed.
2. No high-priority defects remain unresolved.
3. Successful beta testing with user feedback incorporated

8.3 Quality Assurance Plan

In order to make sure AuralAid platform complies with the highest standards of the functionality, usability and reliability, the detailed Quality Assurance (QA) strategy will be applied. This plan contains specific test suites that will address all the essential functionality priorities. The QA procedure will also check on the AI models, the user interaction and flow and on how the system will perform.

8.3.1 Planned Test Suites and Test Cases

The following test suites and test cases are going to be conducted to test the AuralAid ; the descriptions of the functional and nonfunctional requirements are provided below. Every test suite targets one specific feature or ability of the system so that every aspect of it is tested for correctness and stability, speed, and usability.

Test Suite 01: Test Suite for Device Pairing

Table 28: Test Suite for Device Pairing

TC-ID	Title	Description	Steps	Expected Result	Actual Result
TC-1	Successful Device Pairing	Verify the device pairs successfully with the mobile app	1. Power on device 2. Open mobile app 3. Select device 4. Pair	Device connects and displays status in the app	
TC-2	Failed Device Pairing	Test behavior when pairing fails	1. Open app without device powered	App shows error: "Device not found"	

			2. Attempt pairing		
TC-3	Multiple Device Pairing	Test pairing with more than one device	1. Pair first device 2. Pair second device 3. Observe app	Both devices connected and managed separately	
TC-4	Device Disconnection Handling	Verify app handles sudden disconnection	1. Start session 2. Turn off device 3. Observe app	Error message displayed; retry option available	

Test Suite 02: Audio & Speech

Table 29: Test Suite for Audio & Speech

TC-ID	Title	Description	Steps	Expected Result	Actual Result
TC-12	Audio Capture	Verify device captures real-time audio	1. Start audio session 2. Speak near device	Audio captured clearly; minimal delay	
TC-13	Noise Filtering	Background noise removed	1. Speak in noisy environment 2. Observe audio	Noise filtered; speech clear	
TC-14	Speech-to-Text Accuracy	Convert speech to text	1. Speak sentence 2. Check captions on app	Captions match spoken words ≥90%	
TC-15	Real-Time Caption Update	Captions appear instantly	1. Speak continuously 2. Observe captions	Captions updated ≤2s delay	
TC-16	Audio Error Handling	Handle device/audio failures	1. Disconnect device mid-session 2. Observe app	Error displayed; retry option available	

Test Suite 03: Noise Alert / Environmental Safety

Table 30: Test Suite for Noise Alert / Environmental Safety

TC-ID	Title	Description	Steps	Expected Result	Actual Result

TC-17	Noise Alert	Alert when environment noise >85dB	1. Expose device to loud noise 2. Observe alert	Visual/vibration alert triggered	
TC-18	Noise Level Display	Display current noise levels	1. Start session 2. Observe noise meter on app	Noise level accurately displayed	
TC-19	Noise Threshold Adjustment	User adjusts alert thresholds	1. Change threshold to 70dB 2. Generate noise 75dB	Alert triggered at new threshold	

Test Suite 04: Health Monitoring

Table 31: Test Suite for Interview Evaluation

TC-ID	Title	Description	Steps	Expected Result	Actual Result
TC-24	Dashboard Display	Verify all metrics displayed	1. Open dashboard 2. Check audio, health, emotion	All metrics shown correctly and updated in real-time	
TC-25	App Responsiveness	Ensure app works smoothly	1. Navigate Dashboard, History, Settings	No crashes; smooth transitions	
TC-26	History Logging	Track past sessions	1. Complete session 2. Open History tab	Session recorded with correct data	
TC-27	Cloud Data Sync	Sync data to cloud	1. Complete session 2. Check cloud logs	Data uploaded and accessible remotely	
TC-28	Multi-Device Compatibility	App works with multiple devices	1. Pair second device 2. Perform session	Both devices function without conflict	

Test Suite 05: Audio and AI

Table 32: Test Suite for Audio and AI

TC-ID	Title	Description	Steps	Expected Result	Actual Result

TC-29	Emotion Detection	Detect speaker emotions from audio	1. Speak with a specific emotion 2. Check app output	Correct emotion detected ≥80% accuracy	
TC-30	Accent & Language Support	Verify speech-to-text handles multiple accents/languages	1. Speak sentences in English & Roman Urdu 2. Observe captions	Captions accurately reflect speech	
TC-31	Multiple Speaker Detection	Detect and separate multiple speakers	1. Two people speak simultaneously 2. Observe app	Captions separated for each speaker	

Test Suite 06: Noise / Safety

Table 33: Test Suite for Noise / Safety

TC-ID	Title	Description	Steps	Expected Result	Actual Result
TC-32	Sudden Loud Noise Alert	Alert when sudden loud sound occurs	1. Generate 95dB noise near device 2. Observe alert	Instant alert triggered	
TC-33	Threshold Persistence	User-defined thresholds persist across sessions	1. Set threshold 70dB 2. Close & reopen app 3. Generate 75dB noise	Alert triggers at 70dB threshold	
TC-34	Noise Trend Report	Verify noise exposure logged over time	1. Run session 2. Check app history	Noise exposure trends displayed correctly	

Test Suite 07: App UI / Usability

Table 34: Test Suite for App UI / Usability

TC-ID	Title	Description	Steps	Expected Result	Actual Result
TC-42	Settings Update	User updates app/device settings	1. Go to Settings 2. Change settings 3. Save	Settings saved and applied	
TC-43	Notification Settings	User manages alerts/notifications	1. Open Notifications	Notifications follow updated preferences	

			2. Enable/disable alerts 3. Test		
TC-44	Accessibility Features	Verify text size, color, or contrast	1. Change accessibility settings 2. Observe dashboard	UI adapts correctly	

2. Configuration Management Plan

The Configuration Management Plan (CMP) for the Aural Aid project ensures that all software, device firmware, AI models, and project documents are properly tracked, versioned, and maintained throughout the project lifecycle. It defines configuration items (CIs) such as the mobile app source code, device firmware, AI models for speech-to-text and emotion detection, test cases, and documentation, each assigned a unique identifier and version controlled using GitHub with semantic versioning. All changes follow a structured change management process, including formal change requests, review by the team lead or advisor, implementation in development branches, and merging into the main branch after testing, with a detailed change log maintained. Regular configuration audits ensure compliance with design and requirements. Roles and responsibilities are clearly assigned to team members for maintaining, reviewing, and auditing CIs, while tools like Trello or Google Drive support project management and documentation. Overall, this plan ensures the integrity, traceability, and reliability of all Aural Aid project components while supporting smooth collaboration among team members.

Appendix A – Group Log

The table below summarizes the meetings conducted for the project outlining the meeting types, supervisors, and key activities. Each meeting contributed to different stages of the project, including feasibility analysis, proposal design, development, & code evaluations.

Date	Meeting Type	Supervisor	Details
4th October 2025	Feasibility Analysis	Mr. Junaid Ali Khan	Discussed project feasibility, scope, and resources.

11th October 2025	Proposal and Design	Mr. Junaid Ali Khan	Started drafting the proposal, defining project structure and objectives.
18th October 2025	Proposal and Design	Mr. Junaid Ali Khan	Continued proposal drafting, discussed project scope and major deliverables.
25th October 2025	Proposal and Design	Mr. Junaid Ali Khan	Detailed design discussions, including UI/UX requirements and backend architecture.
29th October 2025	Proposal and Design	Junaid Ali Khan	Final review of proposal and design.
8th November 2025	Design	Mr. Junaid Ali Khan.	Designed document
15th November 2024	Design	Mr. Junaid Ali Khan	Continued developing the design framework and scripting.
22nd November 2024	Development	Mr. Junaid Ali Khan	Started development phase, setting up repositories and initial infrastructure.
29th November 2024	Control Management & Configuration	Mr. Junaid Ali Khan	Worked on configuration management tasks, repository setup, and system configurations.
6th December 2024	Code evolution	Mr. Junaid Ali Khan	Evaluated code, feedback mechanisms and optimizing code quality.
13th December 2024	Code Evaluation	Mr. Junaid Ali Khan	Continued evaluating code and refining code evaluation techniques.
18th December 2024	Code Evaluation	Junaid Ali Khan	Finalized code evaluation feedback processes.
21st January 2026	Documentation	Mr. Junaid Ali Khan.	Finalized project documentation and reports.